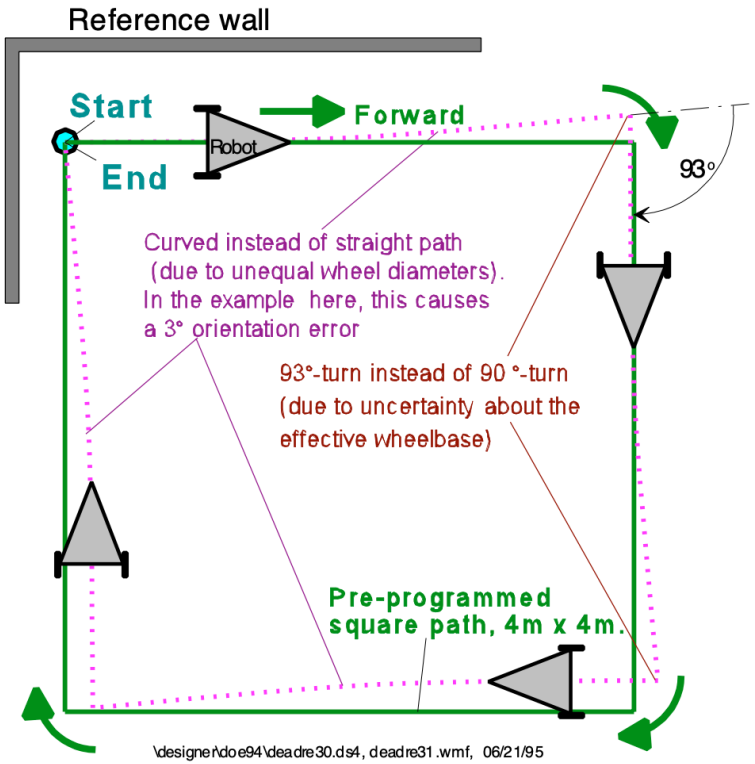
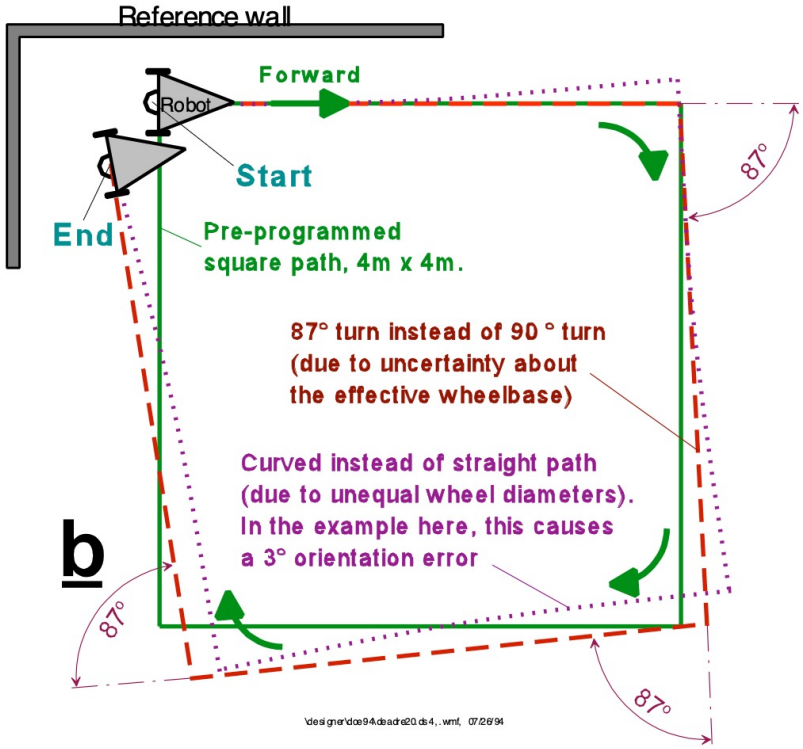
Mobile Robots kinematic model calibration

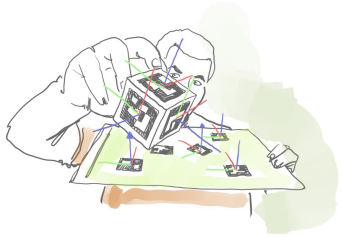
Odometric Calibration

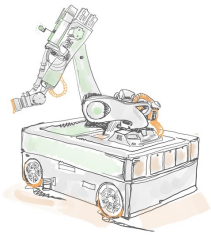




Prof. Mariolino De Cecco
Email: mariolino.dececco@unitn.it

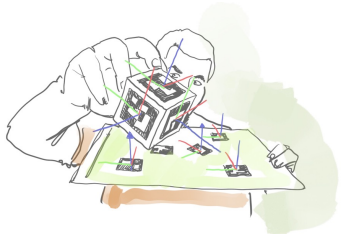
Laboratory website: http://www.miro.ing.unitn.it/

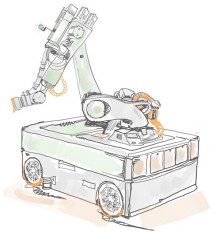




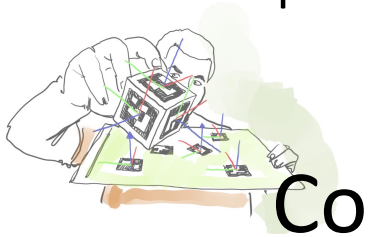
Modeling systematic and random effects

INFLUENCE FACTORS

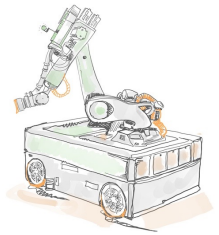




- Influence factors in robotics pose estimation
- Influence factors in measurement
  - interfering/modifying, semi-constant/random
  - Influence factors in kinematic models – the unicycle example: the different effects with a simple matlab model → show the different effects due to random and pseudo-constant factors: from this arise the need for Calibration (e.g. 1° heading error and 1 mm on the radius)
  - Influence factors in other kinematic models: the differential drive and tricycle
  - The dependence from the path: closed path could not reveal some effect
  - The dependence from the numerical integration method
- Calibration:
  - Linear input-output
  - Optimization: some examples
  - Kinematic model calibration using known endpoints (with or without special path that decouple parameter influence?)



Content

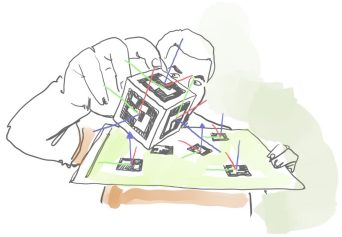


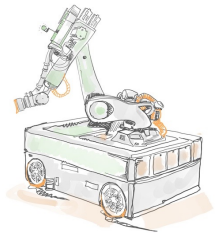
Odometry is the most widely used method for determining the pose (position and attitude) of a mobile robot.
In most practical applications odometry provides easily accessible real-time positioning information in-between periodic absolute position measurements.

The frequency at which the (usually costly and/or time-consuming) absolute measurements must be performed depends to a large degree on the accuracy of the odometry system. It is therefore important for practical mobile robot applications to be aware of the actual odometric accuracy of the platform, in order to space the absolute position updates optimally.

Odometry originates from the Ancient Greek words "hodos" (ὁδός), meaning "way" or "path," and "metron" (μέτρον), meaning "measure»

Therefore, an odometer is literally a "path-measurer"





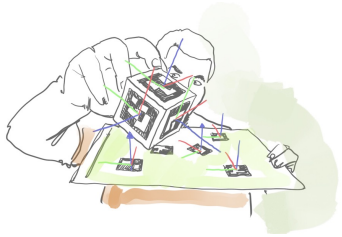
Advantage → ego estimation without any external device/reference. It is only needed the non-holonomous kinematic equations of the robot to estimate pose + usually fast update

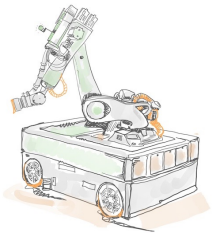
Disadvantage it is inaccurate with an unbounded accumulation of errors → Drift effect due to the **integration** of **non-holonomous constraint (*)**

Typical odometry uncertainties can become so large that the robot's internal position estimate is totally wrong after as little as 10 m of travel

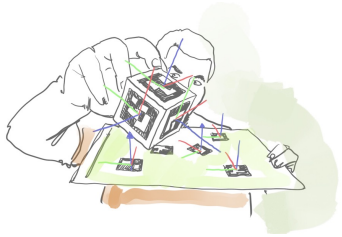
To **reduce** these errors/uncertainties for a feasible industrial or civil application it is needed to perform a good **calibration**

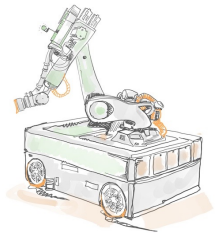
(*): the problem is in wheel robot, not in manipulators!

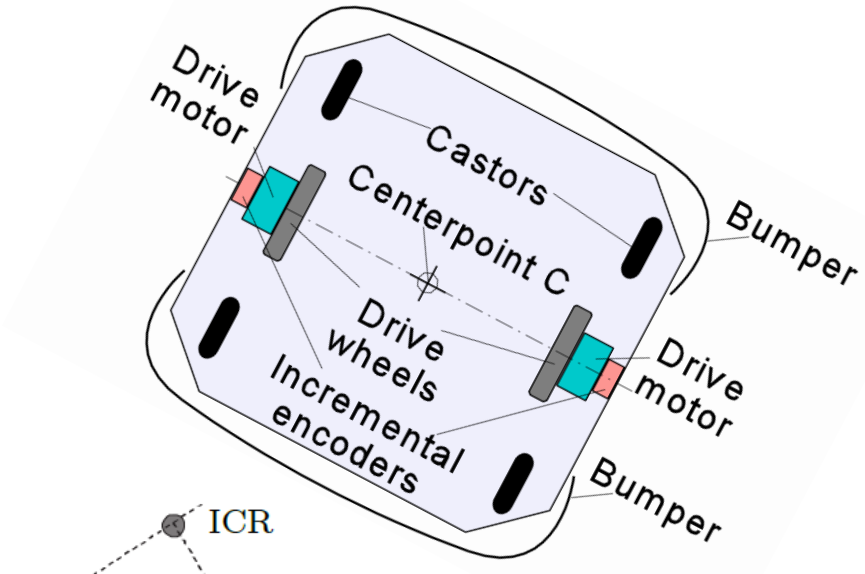


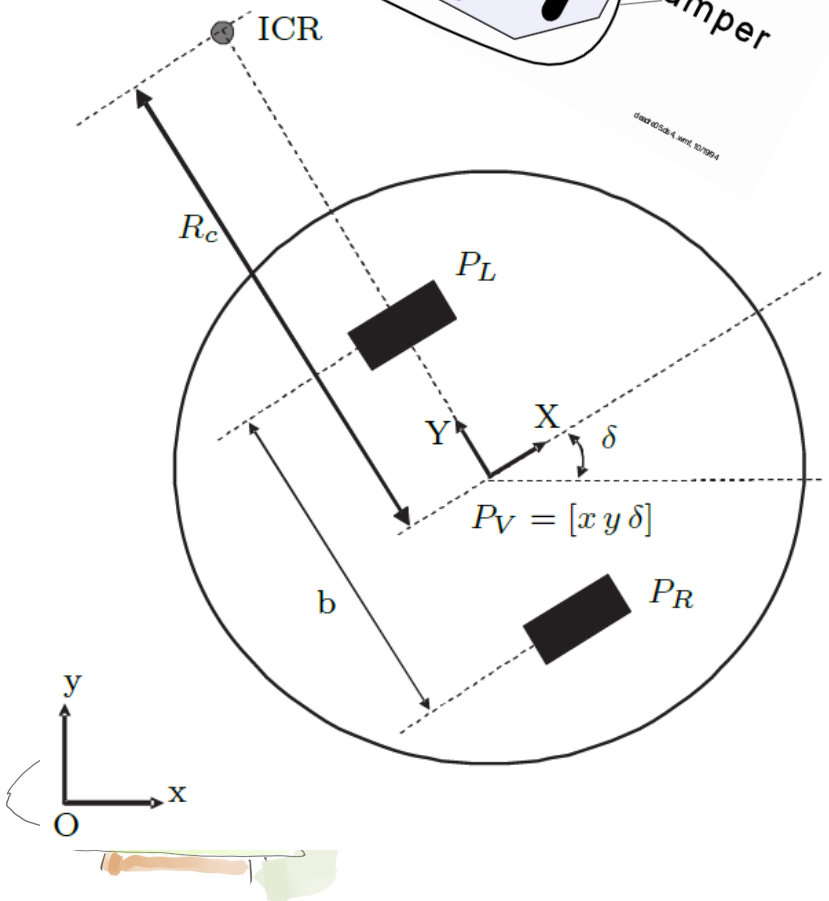


Video AGV odometric path following (Italproject)









In the differential-drive design incremental encoders are mounted onto the two drive motors to count the wheel revolutions.

Using simple geometric equations, it is straight-forward to compute the pose of the vehicle relative to a known starting position.

The basic odometry equations are given

$$\begin{cases} x_{k+1} = x_k + \pi \cdot \dfrac{n_{Rk} \cdot R_R + n_{Lk} \cdot R_L}{n_0} \cdot \cos(\delta_k) \\ y_{k+1} = y_k + \pi \cdot \dfrac{n_{Rk} \cdot R_R + n_{Lk} \cdot R_L}{n_0} \cdot \sin(\delta_k) \\ \delta_{k+1} = \delta_k + 2\pi \cdot \dfrac{n_{Rk} \cdot R_R - n_{Lk} \cdot R_L}{n_0 \cdot b} \end{cases}$$

Where:

R_R is the Right wheel radius

b the robot baseline

n_{Rk} the number of encoder tics for the Right wheel in the control period Tc

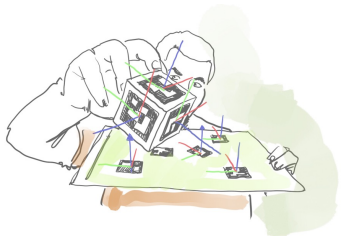
n_0 the number of encoder tics for the whole revolution

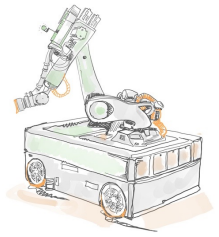
When investigating odometry errors, one should realize that there are two substantially different categories:
Interfering or modifying, quasi-static (systematic) and random.

Below we list all relevant sources of odometry errors according to these two categories:

I. **Systematic (quasi static) errors** are caused by:

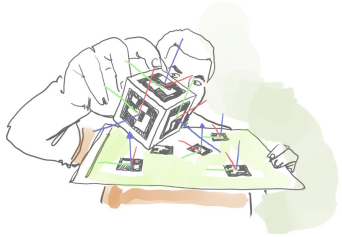
a) Unknown wheel diameters (average of both wheel diameters differs from nominal diameter)
b) Misalignment of wheels
c) Uncertainty about the effective wheelbase due to mechanical construction or simply non-point (or unknown point) wheel contact with the floor

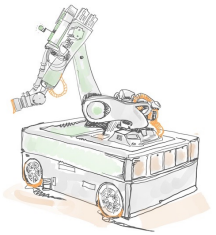




II. **Random errors** can be caused by:

a) Travel over uneven floors
b) Travel over unexpected objects on the floor
c) Wheel-slippage due to:
    - slippery floors
    - over-acceleration
    - fast turning (skidding)
    - external forces (interaction with external bodies)
    - internal forces (e.g., castor wheels)
    - non-point wheel contact with the floor
d) Limited encoder resolution
e) Limited encoder sampling rate





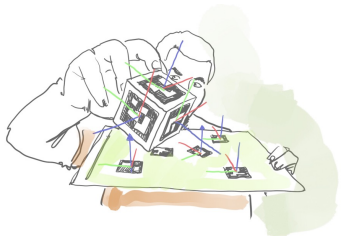
Systematic errors generate strong effects because they, due to integration, accumulate constantly

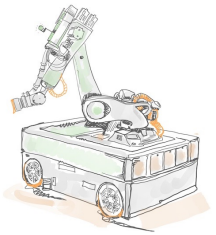
On smooth indoor surfaces systematic errors contribute much more than non-systematic errors

On rough surfaces with significant irregularities, non-systematic errors may be dominant



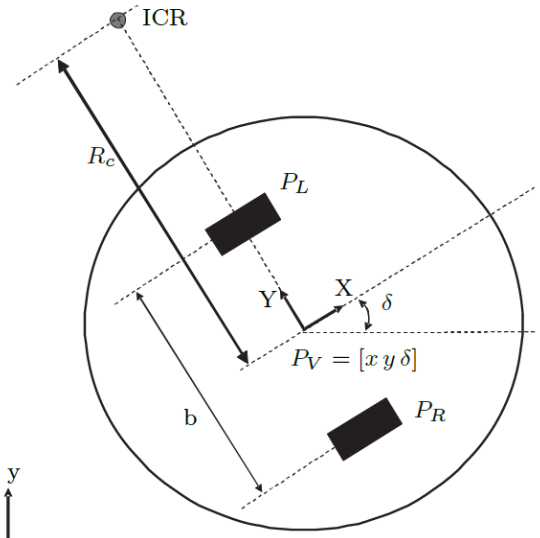


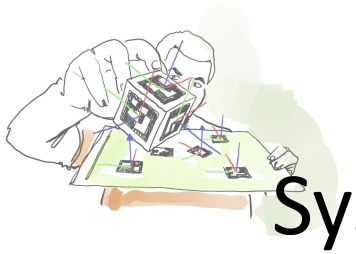


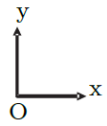


Systematic effects are usually caused by **imperfections in the design** and **mechanical construction** of a mobile robot. In differential-drive robots, two systematic effects:

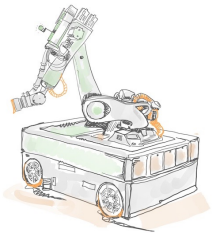
- **Unknown wheel diameters**. Most mobile robots use rubber tires to improve traction. These tires are difficult to **manufacture** to exactly the same diameter.they bear consumption. Furthermore, rubber tires **compress** differently under asymmetric load distribution.
- **Uncertainty about the wheelbase**. The wheelbase is the distance between the contact points of the two drive wheels. Uncertainty is caused by the fact that **rubber tires contact the floor not in one point**, but rather in a contact area. Or **the two wheels are not orthogonal to the floor**. The resulting uncertainty can be on the order of 1% in some commercially available robots.



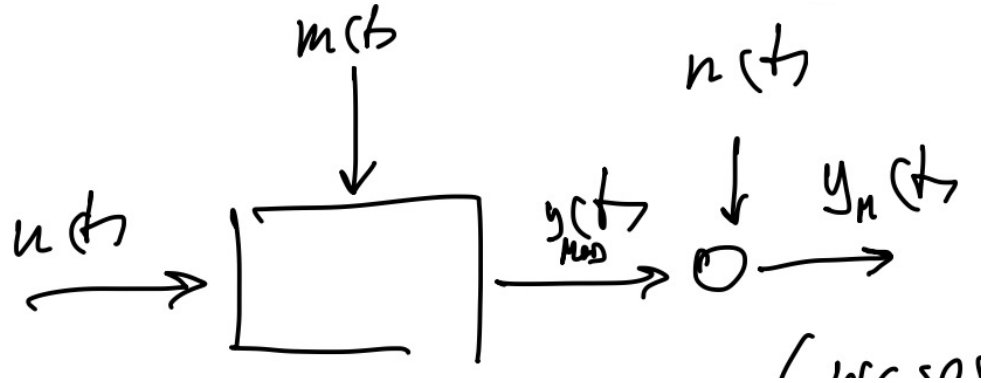




Systematic effects on a differential drive



The modifying and interfering effects are hereafter modelled:



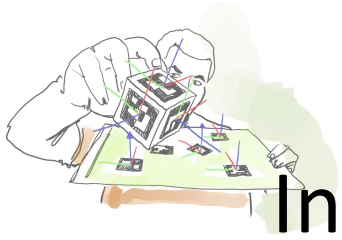
$$y_M(t) = u(t) \cdot (1 + m_0) + n_0 + n(t)$$

Modifying effect

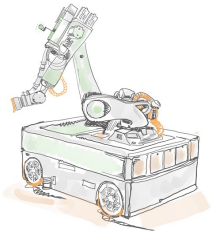
Static (bias)

Dynamic (spike effects, random, …)

Interfering effect



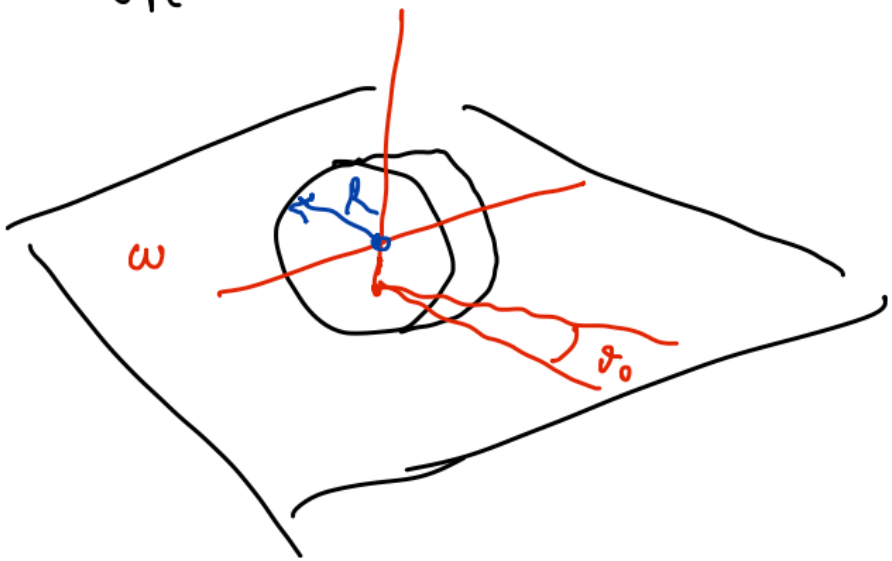
Influence factors in measurement



...:

INTERFERING & MODIFYING EFFECTS IN ROBOTICS – THE UNICYCLE CASE

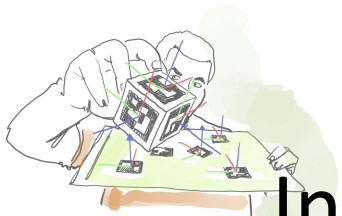
$$y_M = u_{th} \cdot (1 + m_0) + n_0 + n_{th}$$



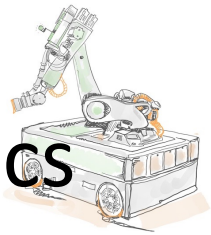
KINEMATIC PARAMETERS:
- $R_K = R + R_0$
- ϑ_0 (ZERO ANGLE)

CONTROL PARAMETERS: (MEASURED)
- ϑ STEERING ANGLE (MEASURED WITH AN ENCODER)
- ω ROLLING ANGULAR VELOCITY (MEASURED WITH AN ENCODER)

$v = \omega \cdot R$ = VELOCITY



Influence factors effects in mobile robotics

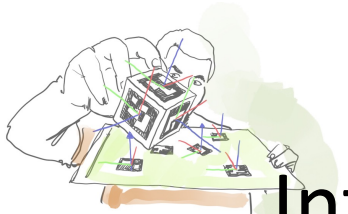


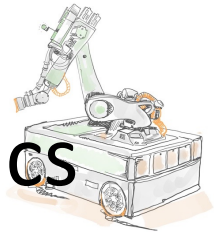
Attitude:

EFFECTS OF THE OFFSET & RANDOM TERMS ON THE REAL AND PERCEIVED PATH

$$\vartheta_M(t) = \vartheta(t) + \vartheta_0 + n_\vartheta(t)$$

M: MEASURED — REAL SENSING — OFFSET OF ZERO ANGLE — RANDOM EFFECT





Influence factors effects in mobile robotics

Forward motion:

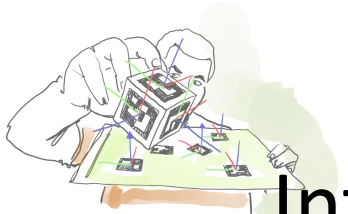
$$\omega_M(t) = \omega(t) + n_\omega(t)$$

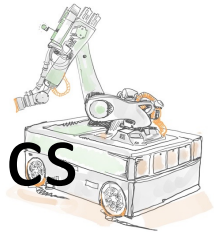
$$v_M(t) = (R + R_0) \cdot (\omega + n_\omega(t)) =$$

$$= R \cdot \omega(t) + (R + R_0) \cdot n_\omega(t) + R_0 \cdot \omega(t) =$$

$$= \underbrace{v(t)}_{\text{REAL VELOCITY}} + \underbrace{R_k \cdot n_\omega(t)}_{\text{RANDOM EFFECT}} + \underbrace{R_0 \cdot \omega(t)}_{\text{OFFSET THAT DEPENDS ON THE ANGULAR VELOCITY (NOT A SIMPLE OFFSET)}}$$

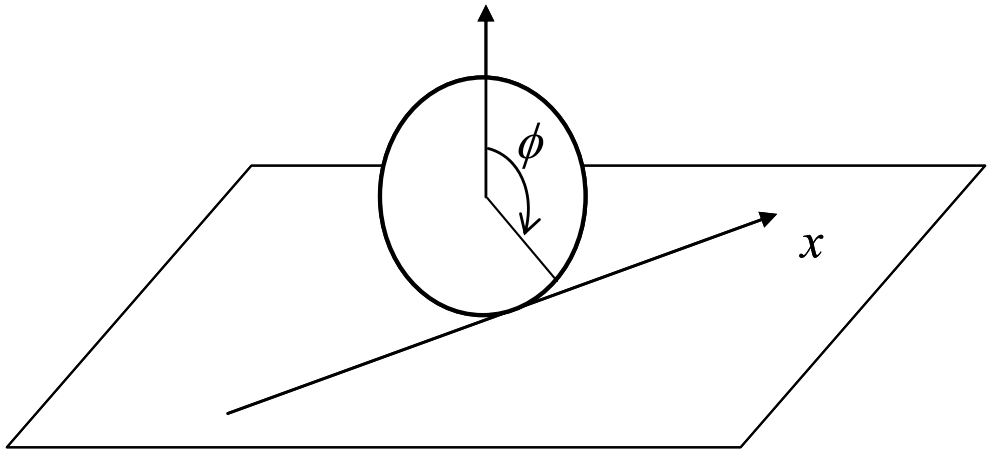
Defined Rk = R+R0

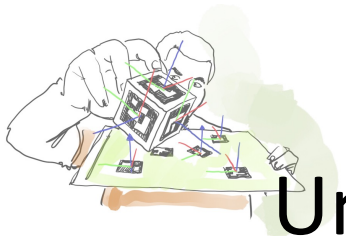




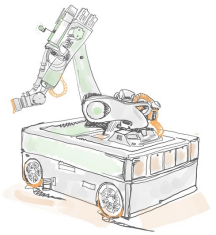
Influence factors effects in mobile robotics

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\delta} \end{bmatrix} = \begin{bmatrix} \cos\delta \\ \sin\delta \\ 0 \end{bmatrix} v + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} w$$





Unicycle model



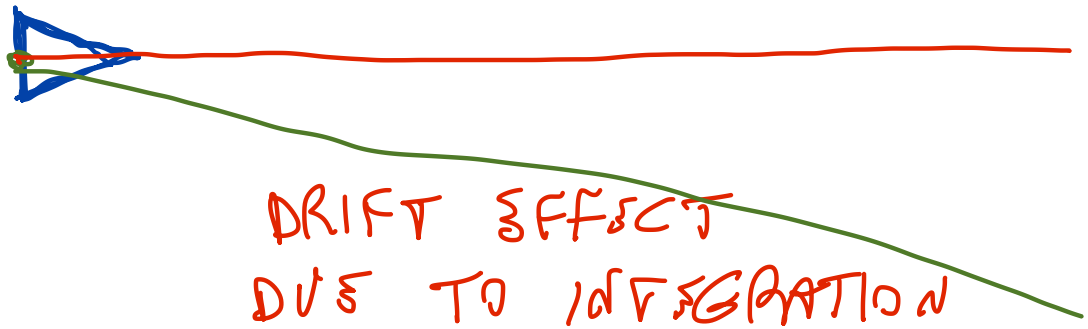
Heading error/steering:

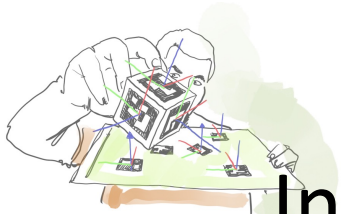
1) CASE OFFSET IN STEERING – STRAIGTH PATH;

$\vartheta_M(t) = \vartheta(t) + \vartheta_0$

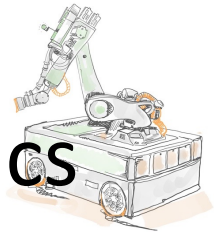
$v_M(t) = v(t)$

THE CONTROL FORCES THE $\vartheta_M \Rightarrow \vartheta(t) = -\vartheta_0$





Influence factors effects in mobile robotics



Heading error/steering

How Precisely Can You Determine a Wheel's Angular Orientation "By Hand"?

The precision with which you can set or measure the angular orientation of a wheel "by hand" (that is, without specialized electronic sensors or measurement tools) is limited and depends on practical factors.

Skill and Tools:

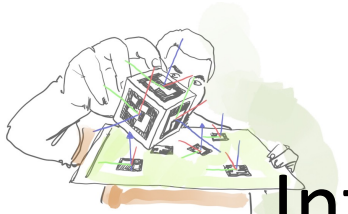
- **With just your eyes and hands, precision is low—typical errors are several degrees, often ±5° or more.**
- Using basic manual tools like a protractor or a precise ruler, you might reduce the error to the range of about 1–2° in ideal conditions, but rarely better.

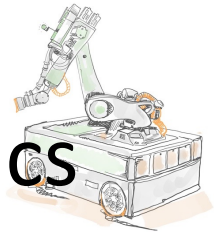
Scale of the Wheel:

- The larger the wheel, the harder it is to judge small angular changes accurately—errors can increase with size.

Observation Method:

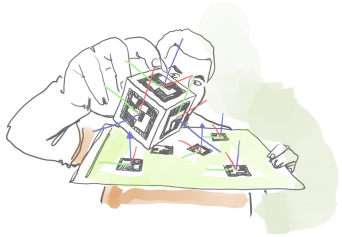
- If you can mark reference lines on the wheel and chassis and use them as a visual guide on a flat surface, accuracy improves. Still, reaching below 1° error by hand is extremely difficult.

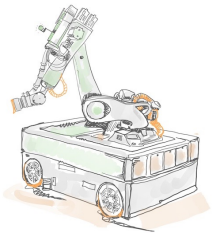




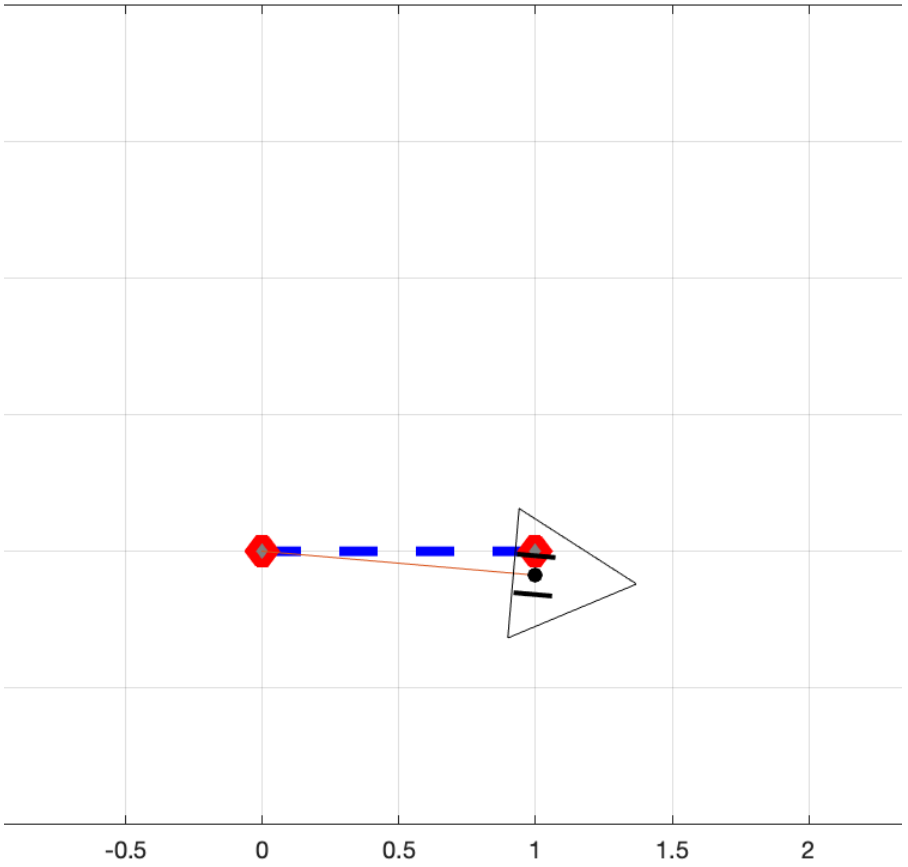
Comparison of Different Methods

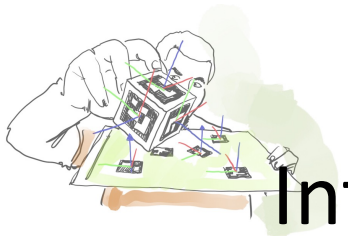
Method	Typical Error Range
By hand, visual estimate (no tools)	±5° or more
With protractor or angle ruler (manual)	±1–2° (in ideal conditions)
With electronic encoders/sensors	Fractions of a degree (e.g., 0.01° or better)



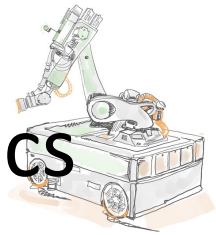


So, imagine to place your wheel aligned with 0 heading by hand without any tool, the heading error could be as much as 5° resulting in 90 mm of deviation. After 10 m would be 900 mm!!!





Influence factors effects in mobile robotics



... 2) CASE OFFSET IN VELOCITY – STRAIGTH PATH:

$\theta_M(t) = \theta(t)$ HERE THE STEERING IS "PERFECT"

$v_M(t) = v(t) + R_0 \cdot \omega(t)$

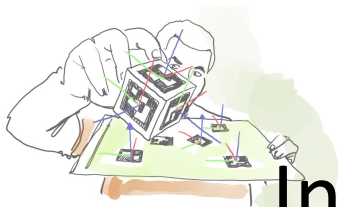


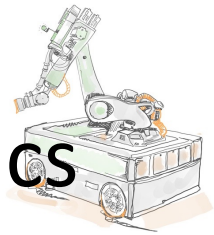
REAL PATH

PERCEIVED PATH

- DRIFT EFFECT DUE TO THE INTEGRATION OF A HIGHER VELOCITY

THIS DEVIATION IS DUE TO THE INACCURATE KNOWLEDGE OF THE WHEEL RADIUS THAT, IF $R_0 \neq 0$, MAKES THE ROBOT PERCEIVE A LONGER PATH DUE TO INTEGRATION





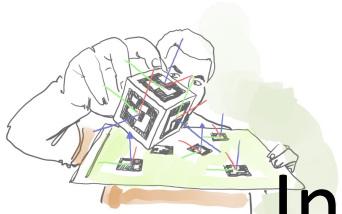
Influence factors effects in mobile robotics

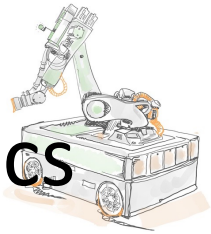
Velocity/radius error:

How Precisely Can You Determine a Wheel's Radius with a Simple Ruler?

Typical uncertainty factors using a simple ruler (without precision instruments like calipers) :

- Resolution (in millimeters)
- Visual alignment: It's often difficult to perfectly align the ruler with the wheel's center and edge—especially by eye—leading to small errors
- geometry of the wheel: could not be perfectly round
- Ability to identify the true center (especially if the hub is covered, irregular, or not clearly marked).
- Wheel edge imperfections (worn rubber, mold lines, etc.).
- Ruler thickness and parallax error (seeing the ruler slightly from the side).

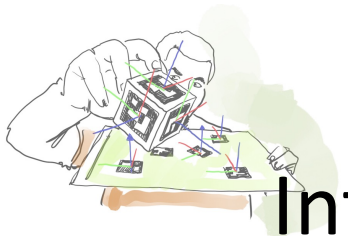




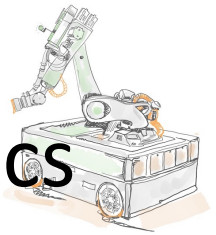
Influence factors effects in mobile robotics

Realistic Error Ranges

Wheel Size	Best-Case Precision (Handheld)	Typical Error
Small wheel (2-10 cm radius)	±0.5–1 mm	Often 1–2 mm
Medium wheel (~10–30 cm radius)	±1 mm	1–3 mm
Large wheel (>30 cm radius)	±2–3 mm	2–5 mm (or more)



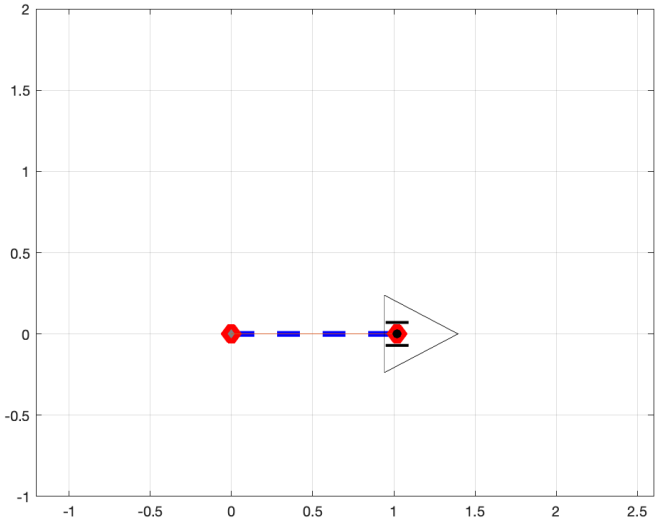
Influence factors effects in mobile robotics

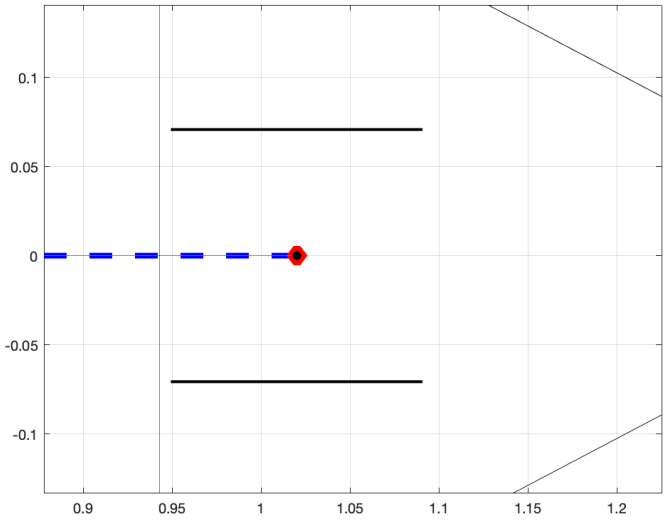


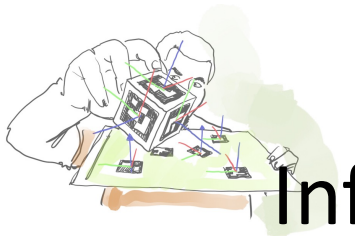
So, imagine to measured the wheel radius around 10 cm with a simple ruler, the measurement error could be as much as 2mm.

The resulting final position error in a path 1 m long would be 20 mm

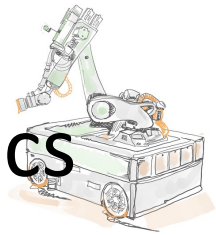
If the path is 10 m long the error would be 200 mm!!!







Influence factors effects in mobile robotics

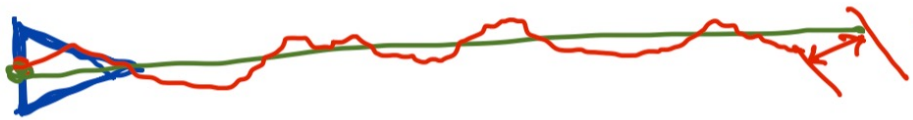


...: 3) RANDOM EFFECTS ON STRAIGHT PATH

SUPPOSE: THE $\vartheta_{th} = 0$ AND $\sigma_{th} = \sigma_0$

$$\vartheta_M(t) = \vartheta(t) + n_\vartheta(t)$$ ← MEAN VALUE = 0

$$\sigma_M(t) = \sigma(t) + R_K \cdot n_w(t)$$ ← MEAN VALUE = 0

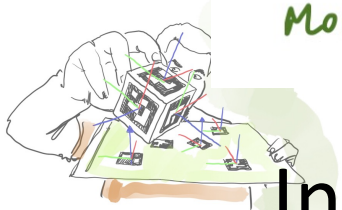


DEVIATION

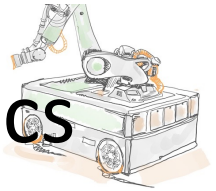
REAL PATH

- DRIFT EFFECT DUE TO THE CHANGE IN DIRECTION AND SAME LENGTH

NOTA: MODELLA BENE IL RUMORE DI UN ENCODER SU n_ϑ e n_w E FAI VEDERE CHE L'EFFETTO È MOLTO LIMITATO? ($\pm$ 1 COUNT)



Influence factors effects in mobile robotics



Random effects in Heading and velocity

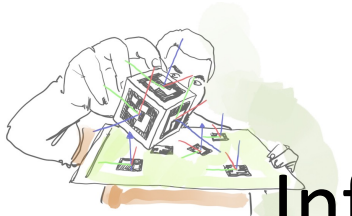
How Precisely Can a robot Determine its heading and rolling velocity?

Typical uncertainty factors depend on measurement :

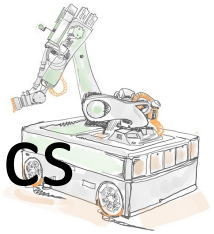
- **Resolution of the encoders** in tic/revolution
- A random effect of $\pm \mathbf{1}\ \boldsymbol{tic}$

A typical wheel encoder resolution can be 4096 [tic/rev]

A typical steering encoder resolution can be 2^14 [tic/rev]



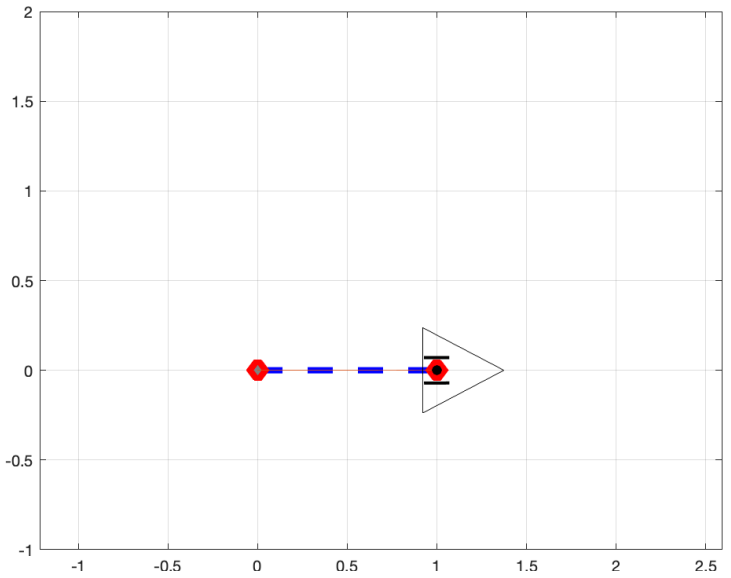
Influence factors effects in mobile robotics

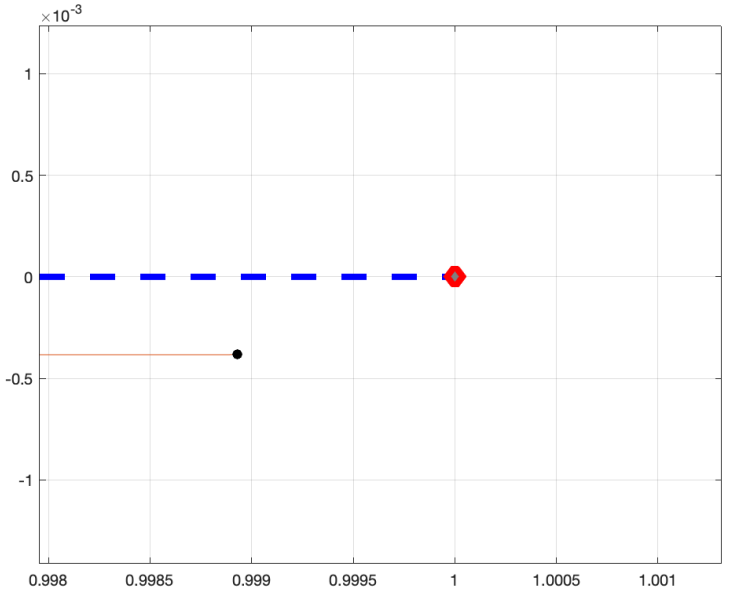


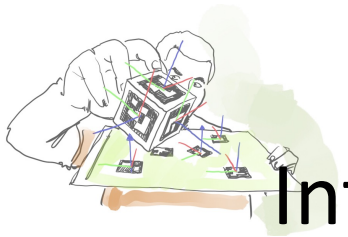
The error after 1 meter is of the order of 0.5 mm

Negligible with respect to the systematic effects in heading and radius/velocity!!!

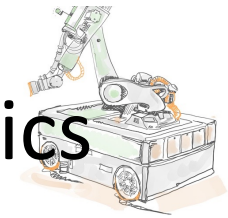
NOTE: the error is in ‘late’ due to the zig-zag effect







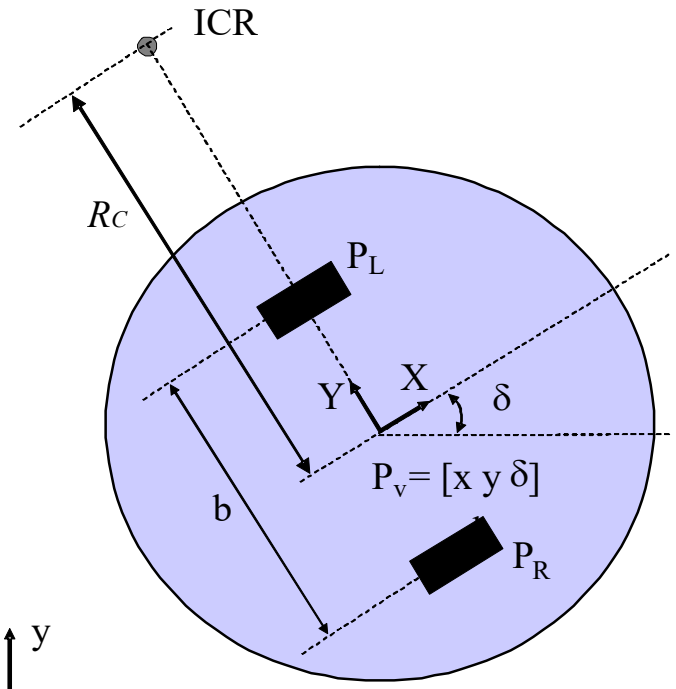
Influence factors effects in mobile robotics



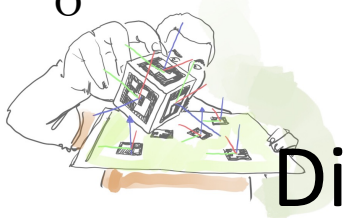
In the differential-drive design incremental encoders are mounted onto the two drive motors to count the wheel revolutions.

Using simple geometric equations, it is straight-forward to compute the pose of the vehicle relative to a known starting position.

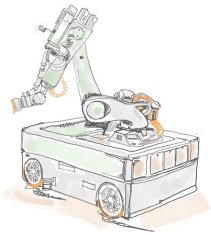
The basic odometry equations are given

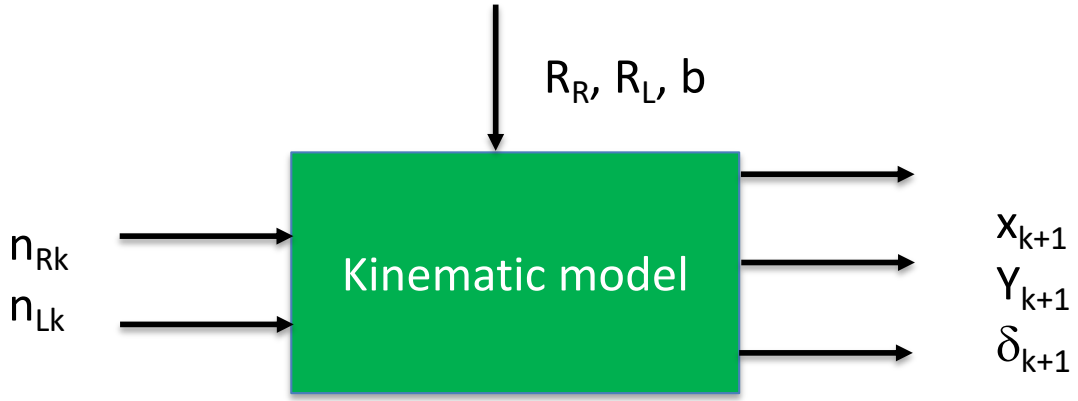


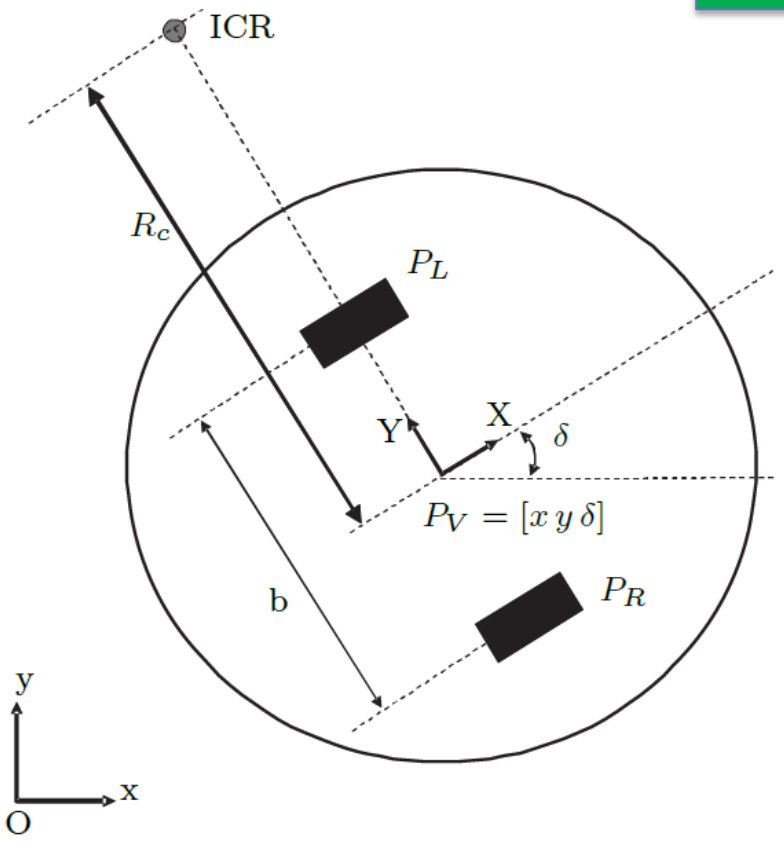
$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\delta} \end{bmatrix} = \begin{bmatrix} \frac{v_R + v_L}{2}\cos\delta \\ \frac{v_R + v_L}{2}\sin\delta \\ \frac{v_R - v_L}{b} \end{bmatrix} = \begin{bmatrix} v\cos\delta \\ v\sin\delta \\ \frac{v_R - v_L}{b} \end{bmatrix}$$



Differential drive model







$$\begin{cases} x_{k+1} = x_k + \pi \cdot \dfrac{n_{Rk} \cdot R_R + n_{Lk} \cdot R_L}{n_0} \cdot \cos(\delta_k) \\ y_{k+1} = y_k + \pi \cdot \dfrac{n_{Rk} \cdot R_R + n_{Lk} \cdot R_L}{n_0} \cdot \sin(\delta_k) \\ \delta_{k+1} = \delta_k + 2\pi \cdot \dfrac{n_{Rk} \cdot R_R - n_{Lk} \cdot R_L}{n_0 \cdot b} \end{cases}$$

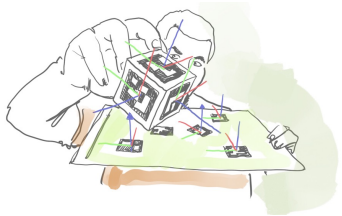
Where:

Inputs: n_{Rk}, n_{Lk} encoder tics for each control loop cycle

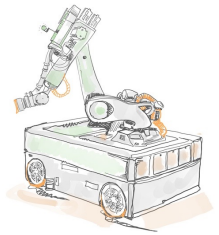
Output: [x,y,δ] is the robot pose

Parameters: R_R, R_L, b radii and baseline

Constant: n_0 is the encoder total tics/revolution

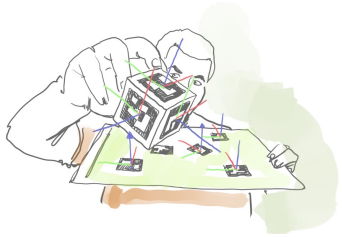


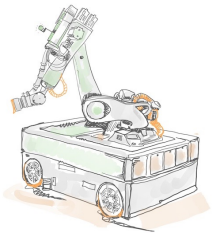
Differential drive discrete model



Simulate a differential drive robot behaviour

MATLAB SIMULATOR

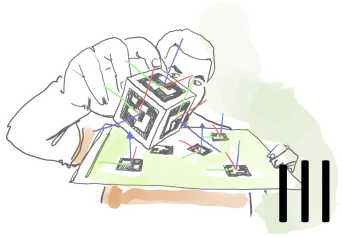




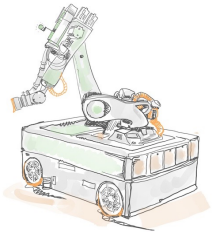
Here let’s go on matlab to practice with a differential drive robot simulator.
Illustrate the files:

- MainDisturbanceEffectsDiffDrive.m

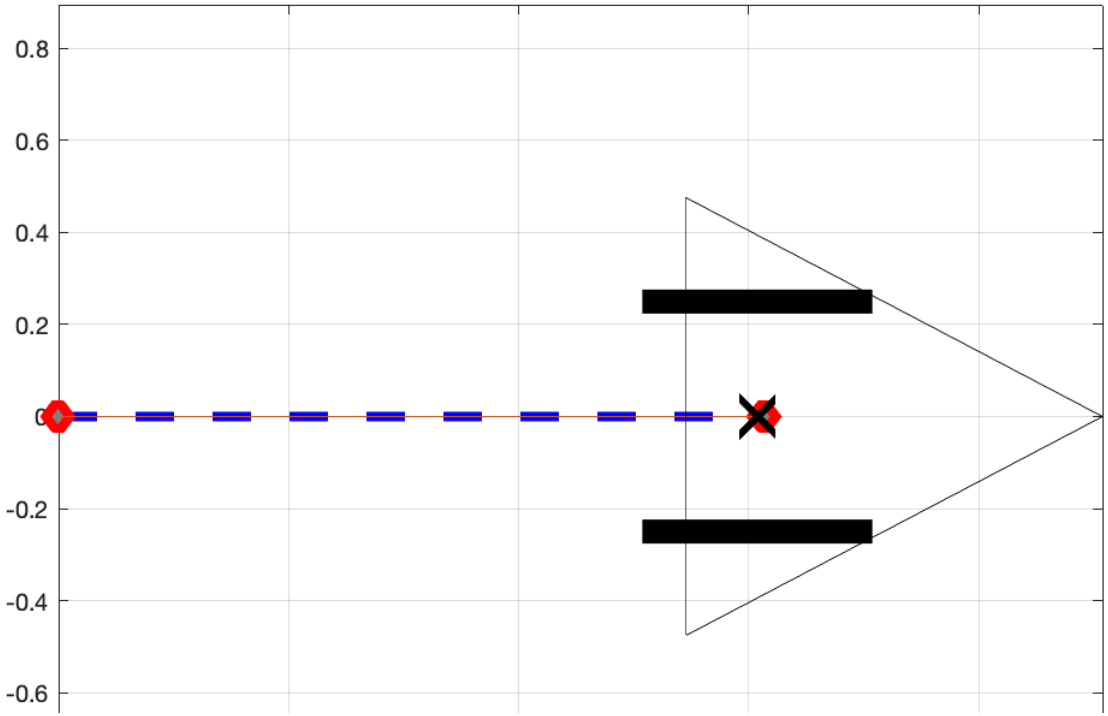
- DiffDriveSimulator.m



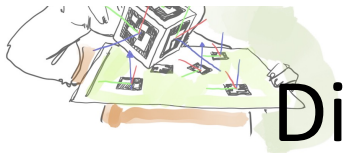
Illustrate the matlab simulator



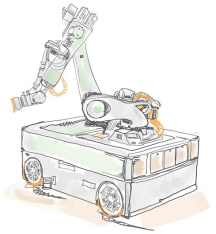
Straight path with ideal parameters



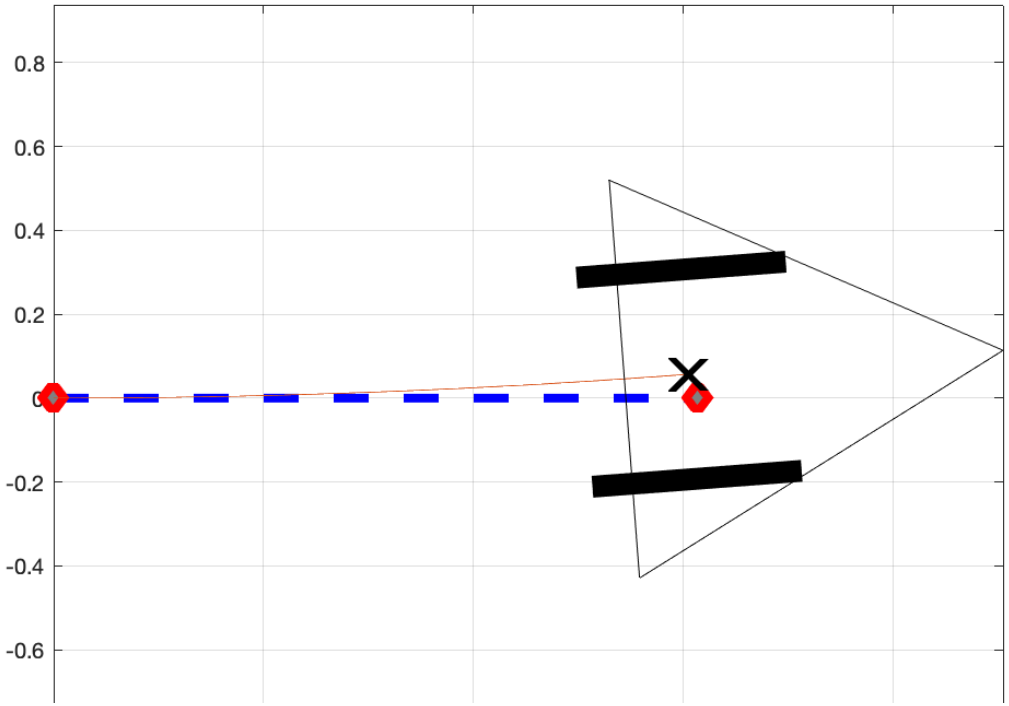

```
% Tic of the encoder wheels
NTic_r = interp1(t, NTic, T, 'linear');
NTic_l = interp1(t, NTic, T, 'linear');
% Kinematic Parameters
K_param = [b R];
DK_param = 0*[0.02 0.001 -0.0015]; % deviation from ideal
parameters
```



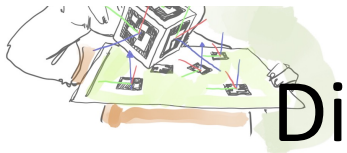
Differential drive simulation



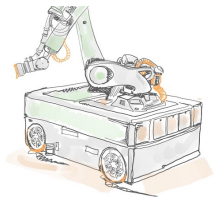
Straight path with non-ideal parameters



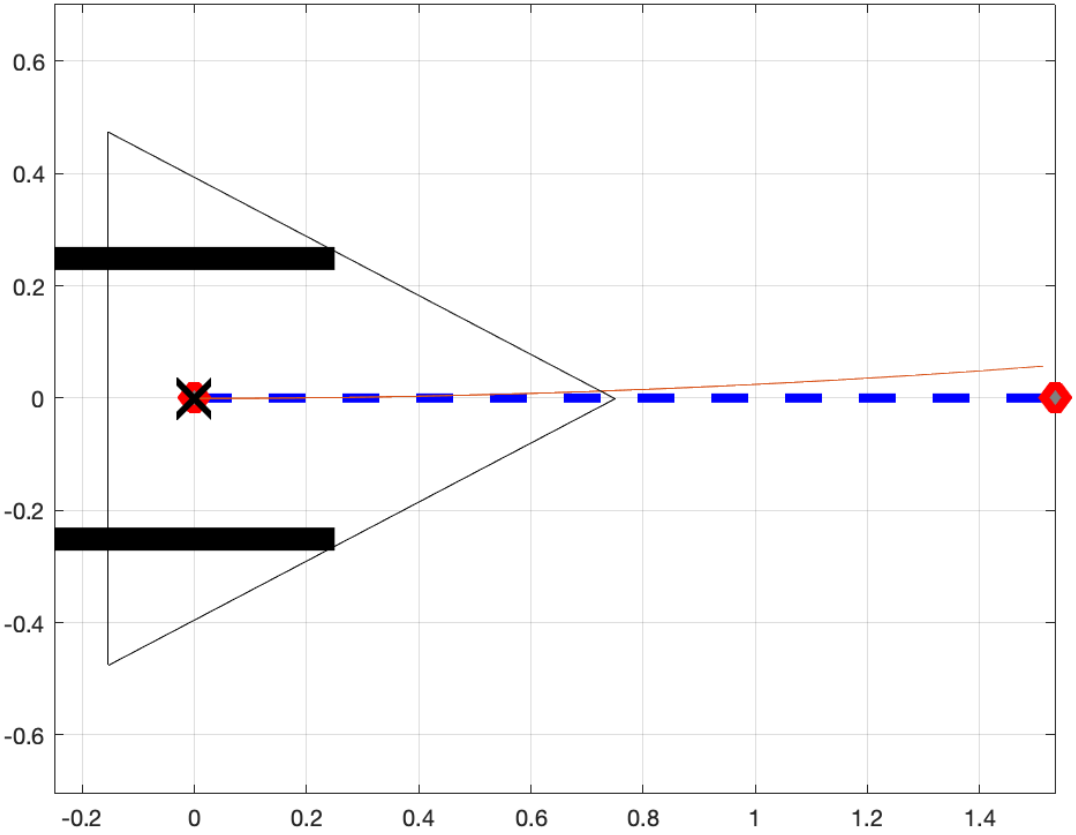

```
% Tic of the encoder wheels
NTic_r = interp1(t, NTic, T, 'linear');
NTic_l = interp1(t, NTic, T, 'linear');
% Kinematic Parameters
K_param = [b R];
DK_param = [0.02 0.001 -0.0015]; % deviation from ideal
parameters
```



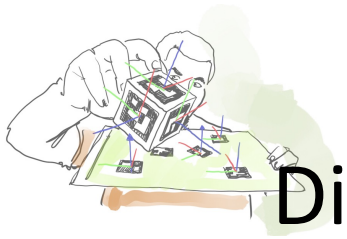
Differential drive simulation



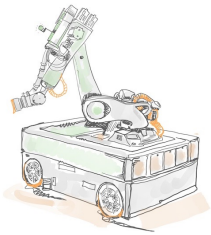
What happen if the robot travel a certain path then comes back?

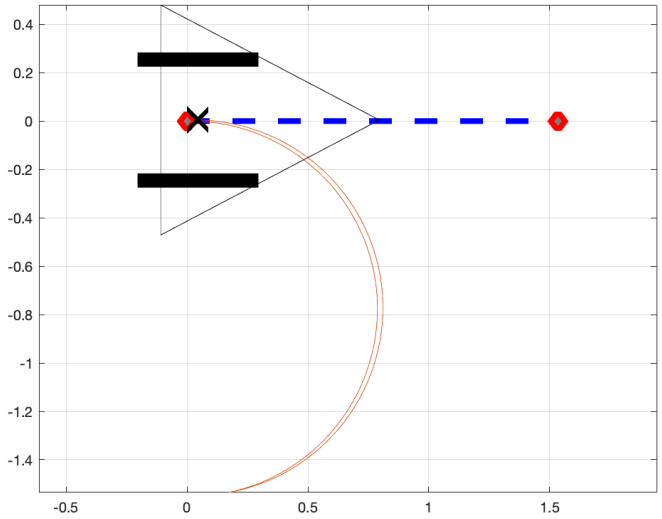


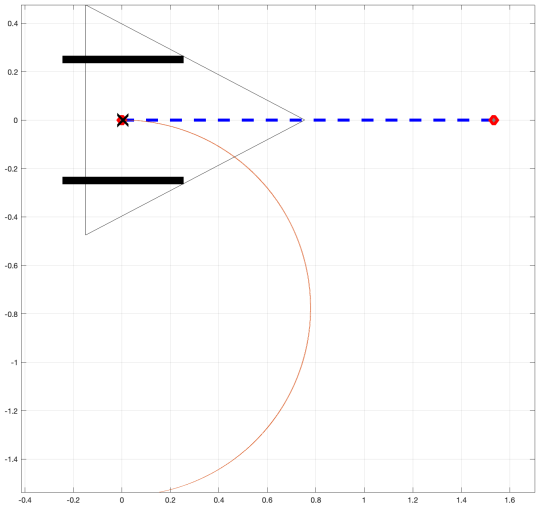
The final position error compensates and thus the kinematic parameters error do not have effect!!



Differential drive simulation





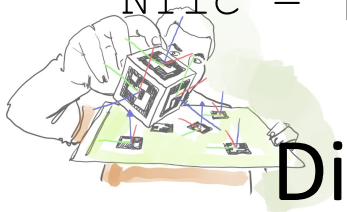


```
% Define the While Loop
update time [s]:
Tc = 0.05;
% Define the current time
[s]:
T = 0;

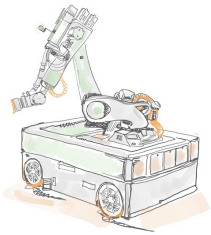
NTic = 100 * ones(1,100);
NTic = [NTic -NTic]
```

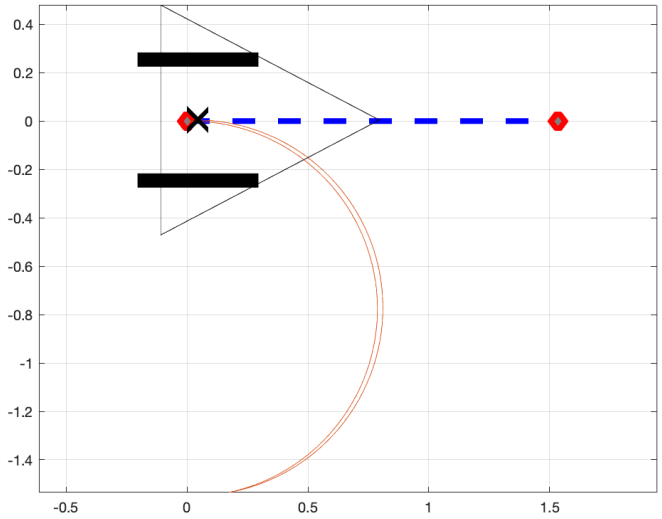
```
% Define the While Loop
update time [s]:
Tc = 0.005;
% Define the current time
[s]:
T = 0;

NTic = 10 * ones(1,1000);
NTic = [NTic -NTic]
```



Differential drive simulation

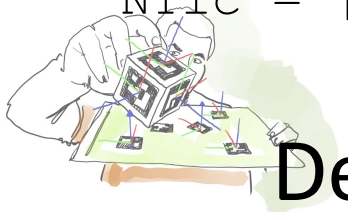




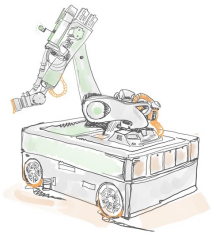
Why there is this deviation while going and coming back?

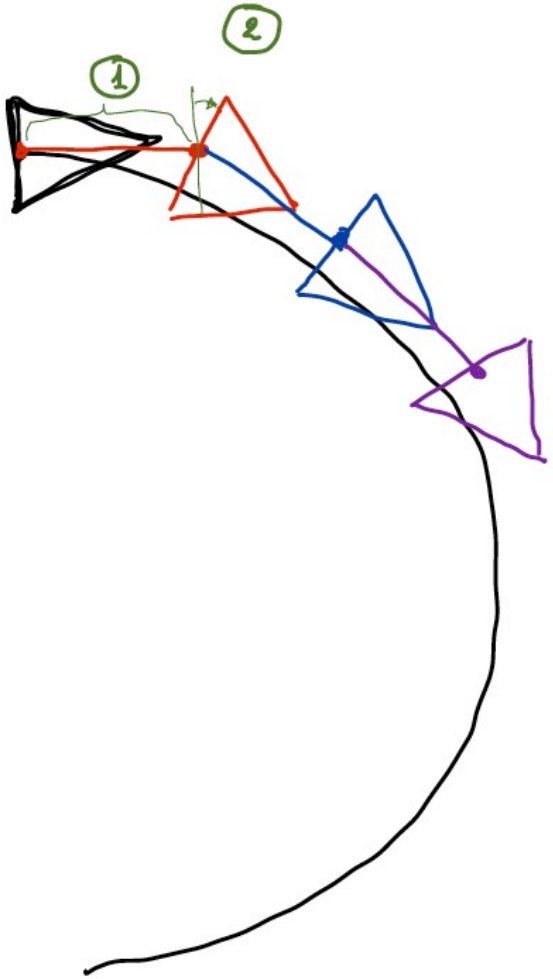
```
% Define the While Loop
update time [s]:
Tc = 0.05;
% Define the current time
[s]:
T = 0;

NTic = 100 * ones(1,100);
NTic = [NTic -NTic]
```



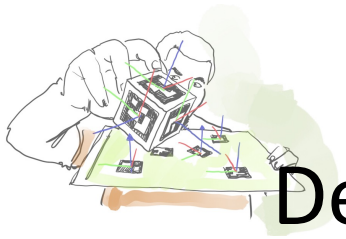
Deviation in simulation



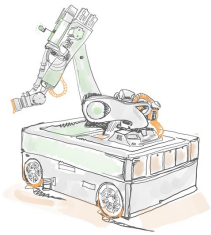


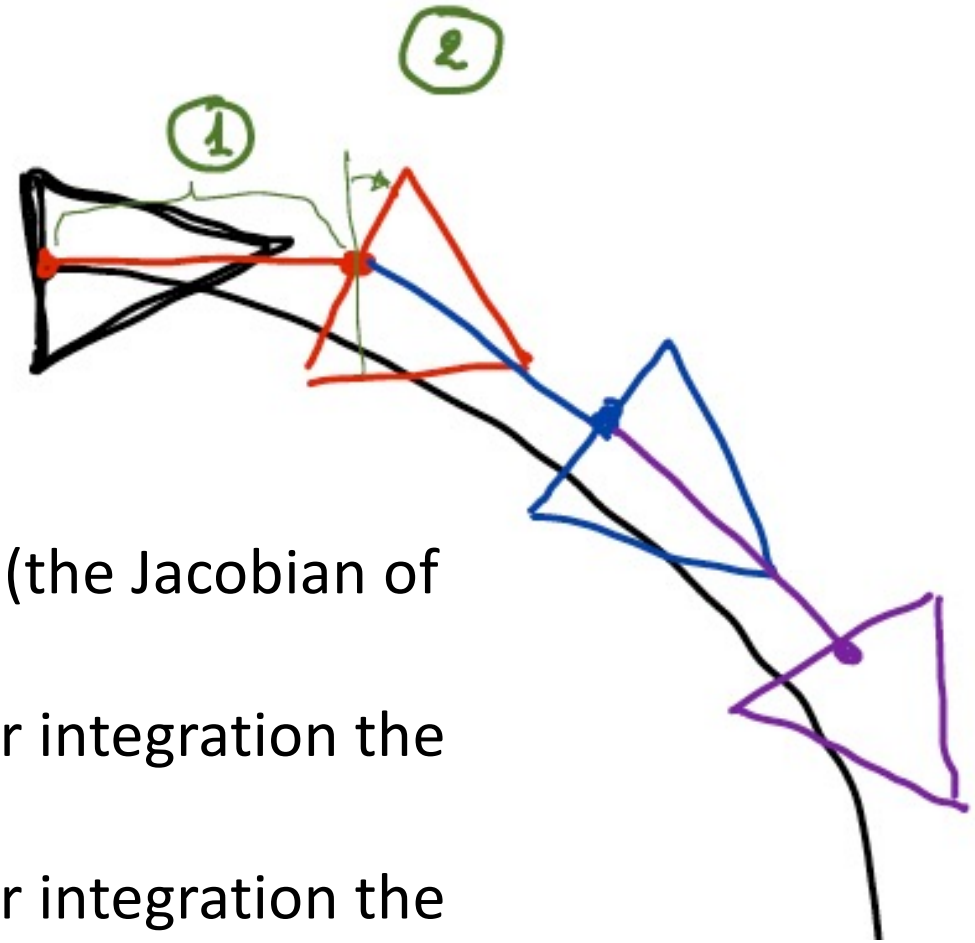
The deviation arises from two main reasons:

- Strapdown sequence
- First order discrete integration
- Impossibility to reverse actions



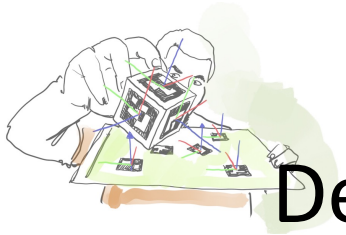
Deviation in simulation



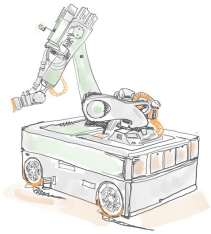


Strapdown sequence:

- First compute derivatives (the Jacobian of the kinematic model)
- Then estimate via 1° order integration the change in position
- Then estimate via 1° order integration the change in attitude
- … Repeat until the end



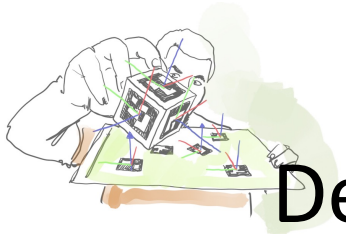
Deviation in simulation - strapdown



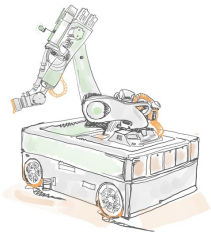
```
% kinematic simulation with l/r encoders Tic for each cycle
function SimulateEnc(obj, dt, NTic_r, NTic_l)
b = obj.K_param(1);
current_pose = obj.robotCurrentPose;

Vr = (obj.K_param(2) + obj.DK_param(2)) * 2*pi * NTic_r /
(obj.Enc_res * dt);
Vl = (obj.K_param(3) + obj.DK_param(3)) * 2*pi * NTic_l /
(obj.Enc_res * dt);

% Diff Drive Model:
% [   xdot   ]   [ cos(delta)/2     cos(delta)/2 ]  [  Vr  ]
% [   ydot   ] = [ sin(delta)/2     sin(delta)/2 ]  [  Vl  ]
% [ deltadot ]   [     1/b          -1/b ]
J = [cos(current_pose(3))/2 cos(current_pose(3))/2;...
     sin(current_pose(3))/2 sin(current_pose(3))/2;...
     1/b                    -1/b];
derivative_pose = J * [Vr; Vl];
xdot = derivative_pose(1);
ydot = derivative_pose(2);
deltadot = derivative_pose(3);
```



Deviation in simulation - strapdown



1

```
x_new = current_pose(1) + xdot * dt;
y_new = current_pose(2) + ydot * dt;
```

2

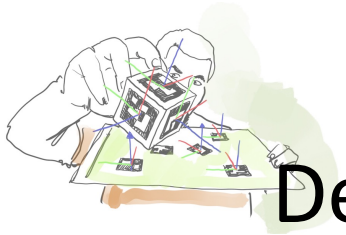
```
delta_new = current_pose(3) + deltadot * dt;

obj.robotCurrentPose = [x_new, y_new, delta_new];
obj.actualPath = [obj.actualPath; x_new, y_new];

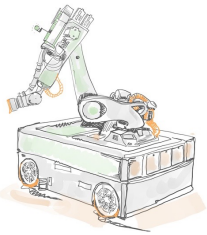
end
```

1: In computing the new position [x y] uses the actual (old) pose (Delta)

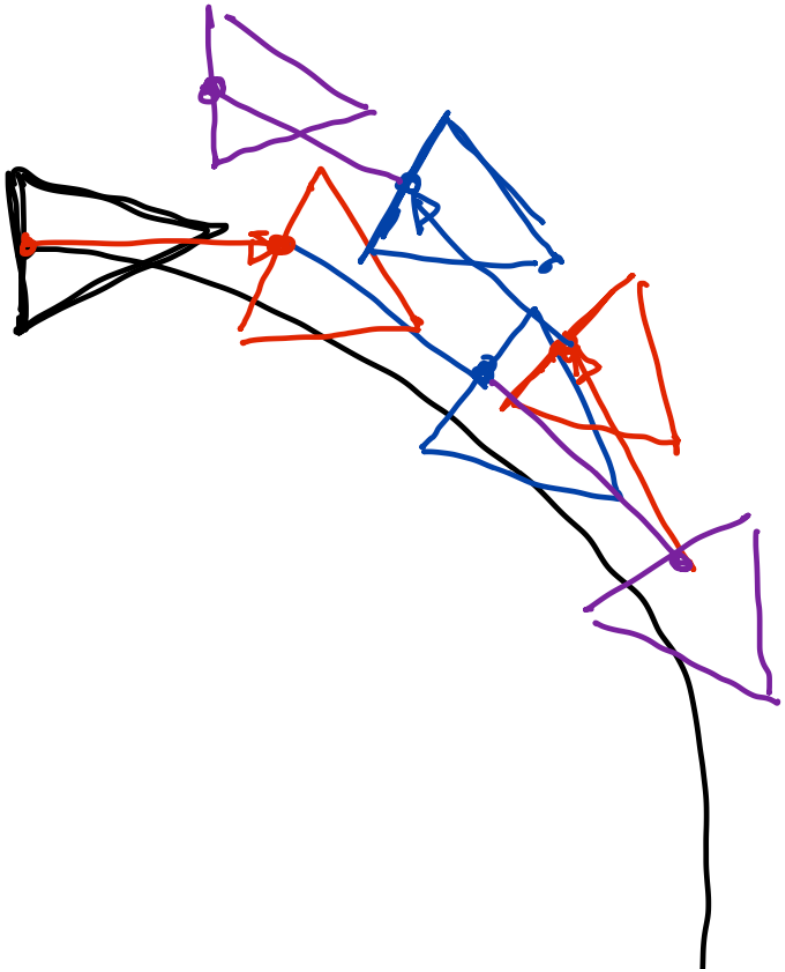
2: the new pose (Delta: `robotCurrentPose(3)`) is computed in this step but used to estimate position in the following cycle

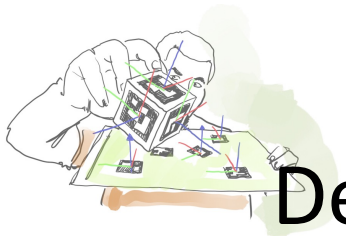


Deviation in simulation - strapdown

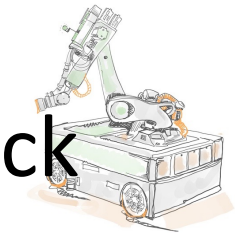


- To get the robot to exactly **retrace** its previous path in reverse (to return to its starting pose):
- You must **reverse both the direction and the order** of the inputs over time
- That is: you must “press play” *backwards* on your commands rather than simply providing negative velocities





Deviation in simulation – actions playback



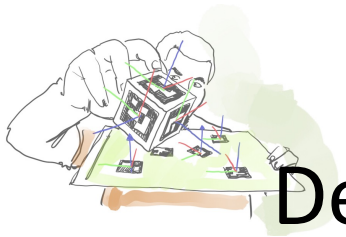
- To verify the effect at different discretisation steps try to change the following:

```
% Define the While Loop update time [s]:
Tc = 0.05;
% Define the current time [s]:
T = 0;

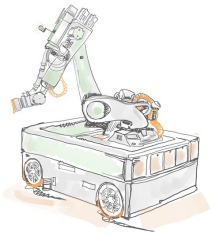
NTic = 100 * ones(1,100);
NTic = [NTic -NTic];
```

Increase Tc n times
And NTic accordingly

The errors will also vary according to the amount of discretisation in time

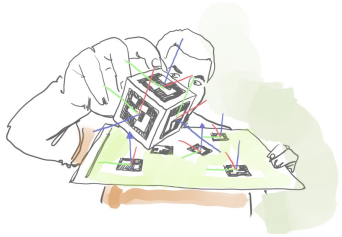


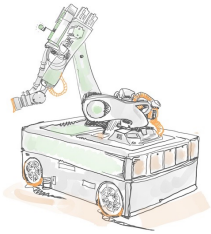
Deviation in simulation



Simulate the robot behaviour according to different l/r wheel inputs

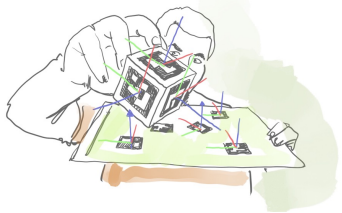
PRACTICE IN CLASS



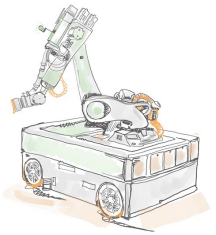


Try:

- **Changing the heading error: theta0 = theta0 + 5*pi/180;**
- **Changing the kinematic parameters (first one radius, then the baseline):**
  - `% Baseline`
  - `b = 0.5;`
  - `% Wheel radii`
  - `R = [0.1 0.1];`
  - `% Kinematic Parameters`
  - `K_param = [b R];`
  - `DK_param = [0.02 0.02 0]; % deviation from ideal parameters`
- **Changing the path from straight to a perfect circle of radius 1m with:**
  - `DK_param = [0 0 0];`
- **Changing the kinematic parameters (first the baseline, then one wheel radius):**
- **Changing the path to a perfect 8 of radius 1m**
- **Make the robot accomplish a squared path of 1m edge**

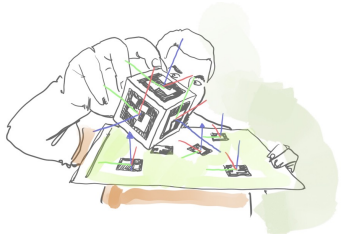


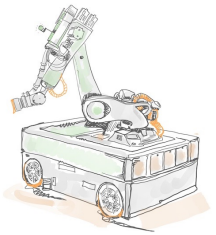
Practice in class



Estimating systematic effects

CALIBRATION

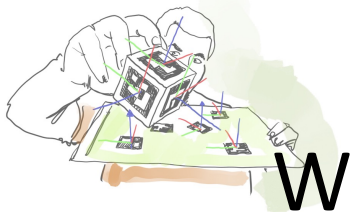




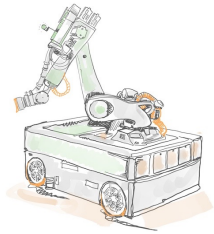
- **Calibration is the process of determining and compensating systematic errors** in a measurement model, so that its output matches a known reference or ground truth as closely as possible

Calibration in Mobile Robotics:

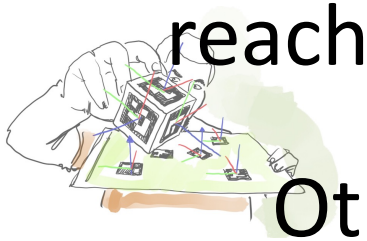
- **Sensor calibration**: Ensures that data from wheel encoders, cameras, LIDAR, or IMUs accurately reflect the robot's real motion and environment by finding and correcting: measurement biases, drifts, scale factors, relative pose (sensor-robot)
- **Kinematic model calibration**: Adjusts parameters like wheel diameters, wheelbase, or offsets in odometry models to minimize the difference between predicted and observed robot position



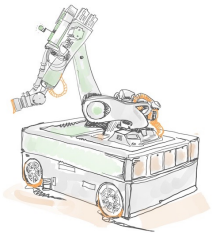
What is Calibration?



- **Camera calibration**: Estimating intrinsic parameters (focal length, lens distortion) and extrinsic parameters (camera pose wrt the robot) for accurate and referenced vision
- **Force/torque sensor calibration**: Adjusts sensor output for zero-drift and gain errors in robotic arms
- **Robot arm calibration**: Tunes Denavit Hartemberg parameters or joint angle offsets to improve end robot accuracy in reaching/following trajectories



Other Common Calibration Examples

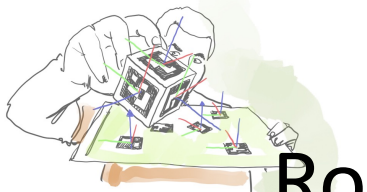


- Optimization methods are essential for calibration: They are used to automatically estimate/correct model parameters by minimizing the error between predicted sensor/model output and true values (estimated via reference sensors)

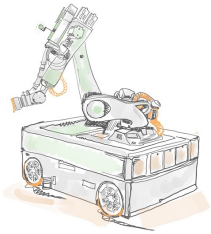
Typical process:

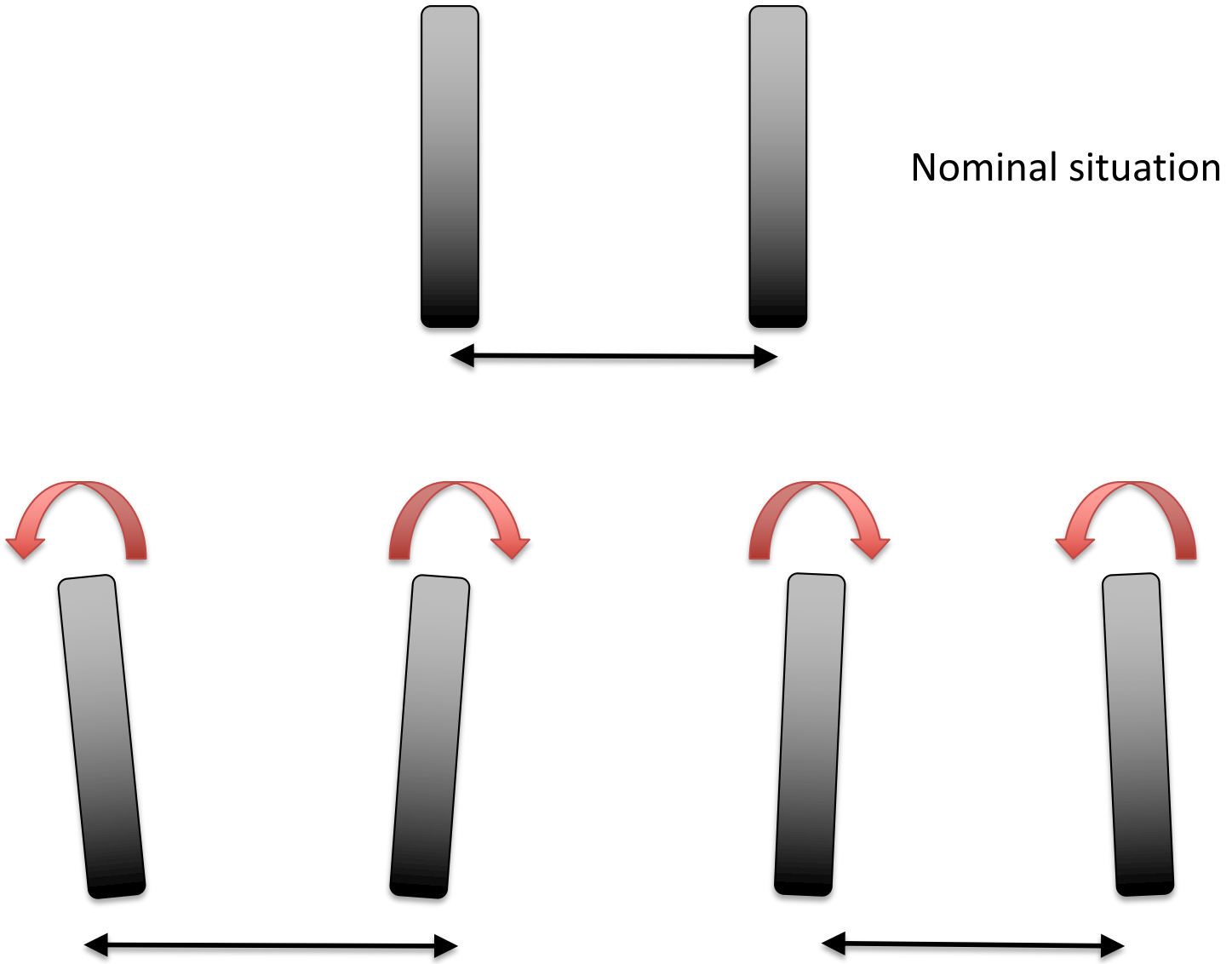
1. Define a **cost function** (e.g., the sum of squared errors between measured and estimated values)
2. use an **optimization algorithm** (like least squares, gradient descent, or more advanced techniques) to find the parameter values that minimize this error

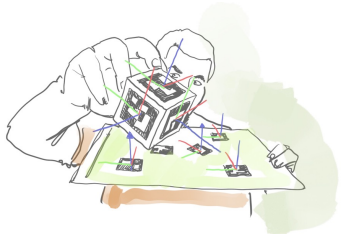
Advantage: Enables systematic, data-driven, and repeatable calibration, even when direct measurement is impossible or when there are many interdependent variables



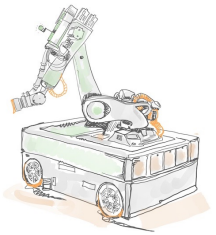
Role of Optimisation Methods







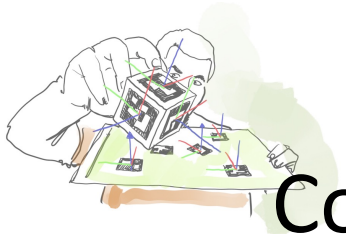
An example – the baseline



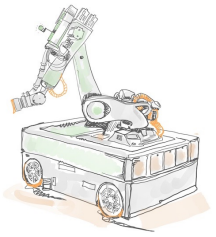
- Estimation of Linear input-output
- Estimation of non-linear input-output

- Optimization: some examples and general framework for engineering problems

- Kinematic model calibration using known endpoints (with or without special path that decouple parameter influence?)



Content



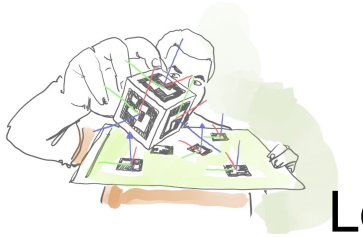
The linear least squares problem is a computational problem of primary importance, which originally arose from the need to fit a linear mathematical model to given observations.

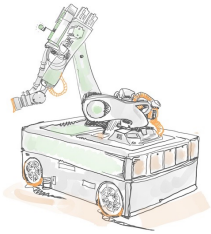
In order to reduce the influence of uncertainty in the observations one would then like to use a greater number of measurements than the number of unknown parameters in the model.

The resulting problem is to "solve" an overdetermined linear system of equations.

In matrix terms, given a vector $b \in R^m$ and a matrix $A \in R^{mxn}$, $m > n$, we want to find a vector $x \in R^n$ such that $A \bullet x$ is the "best" approximation to b so that:

$$A \cdot x \cong b$$





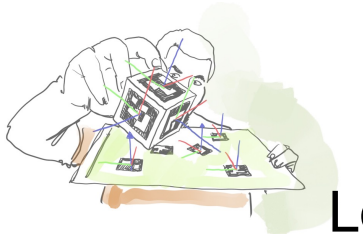
There are many possible ways of defining the "best" solution. A choice which can often be motivated for statistical reasons is to let x be a solution to the minimization problem

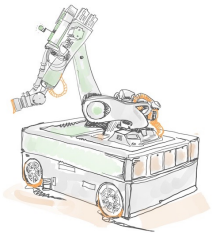
$$\min_x \|A \cdot x - b\|_2 \quad A \in R^{m \times n} \quad b \in R^m$$

We refer to r = Ax - b as the residual vector

A least squares solution minimizes the residual vector 2-norm (the sum of the squared components)

It is to note that, if rank (A) < n, then the solution x is not unique. In this case the problem is underdetermined.





Theorem: Ortogonality condition:

$$A^T \cdot (b - A \cdot x) = 0$$

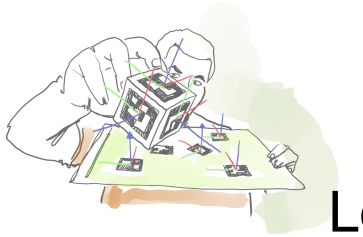
$$A^T \cdot b = A^T A \cdot x$$

This can be inverted!

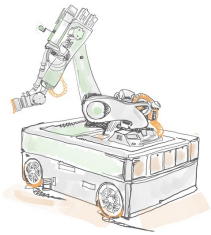
From Theorem it follows that if **rank *(A)* = *n***, then the unique least squares solution *x* and the corresponding residual $r = b - Ax$ are given by:

$$x = \left(A^T A\right)^{-1} A^T \cdot b$$

$$A^{\psi} = (A^T \cdot A)^{-1} A^T$$

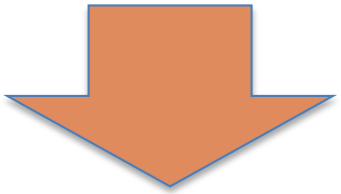


Least Squares problem – the **Pseudoinverse**



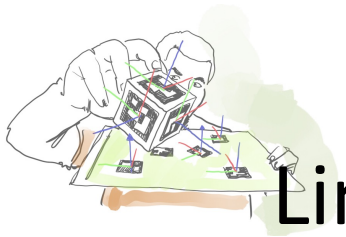
$$A \cdot x \cong b$$

Linear model Experimental data

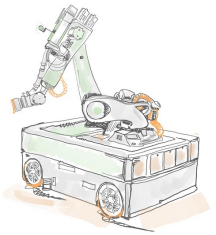


Being not equal the relation can only be made similar and thus their difference minimised

$$\min_x \left\| A \cdot x - b \right\|_2 \quad A \in R^{m \times n} \quad b \in R^m$$



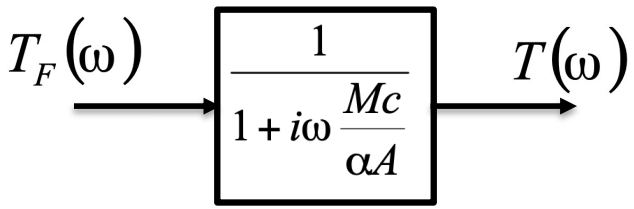
Linear models calibration



In the case of a temperature probe that measures the temperature of a fluid $T_F(t)$, the transfer function can be modelled via a first order dynamical model:

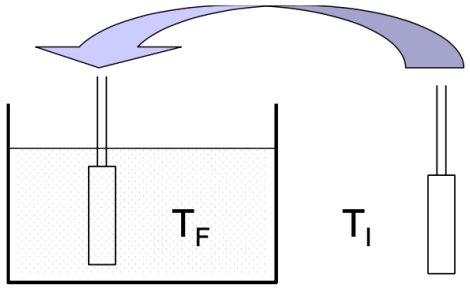
$$H(\omega) \equiv \frac{T(\omega)}{T_F(\omega)} = \frac{1}{1 + i\omega \dfrac{Mc}{\alpha A}}$$

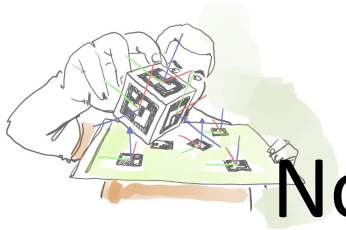
Here the ratio $\dfrac{Mc}{\alpha A}$ is a time that we can define τ and represent the unknown parameter



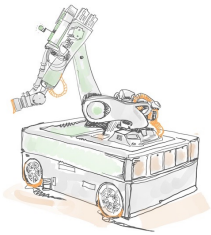
If we consider the case of a PT100 probe suddenly immersed in a thermostat bath, the probe temperature starting at T_i reaches that of the bathroom T_f according to the law:

$$T(t) = (T_i - T_f)e^{-\frac{t}{\tau}} + T_f$$





Non-Linear models calibration



$$T(t) = (T_i - T_f)e^{-\frac{t}{\tau}} + T_f$$

This temperature behaviour is exponential and, therefore, non-linear.

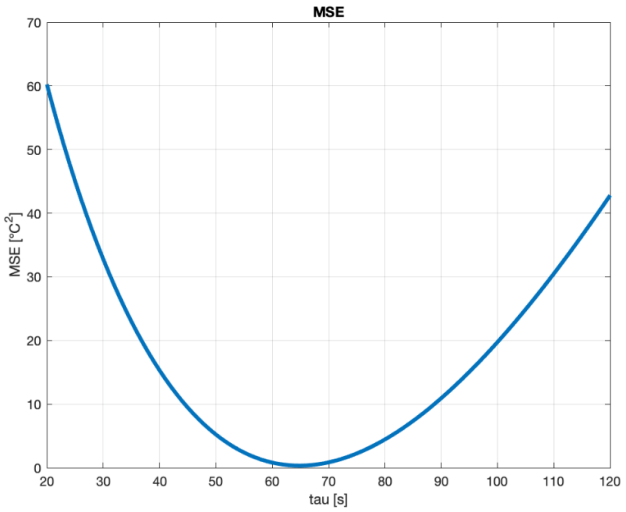
Similarly to the linear case, if we collect experimental data, we obtain:

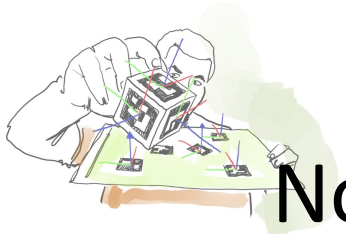
$$T(k \cdot T_c) = (T_i - T_f) \cdot e^{-\frac{kT_c}{\tau}} + T_f$$

Experimental data Non-Linear model

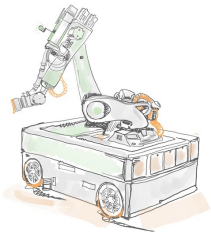
Being not equal the relation can only be made similar and thus their difference minimised

$$Min \left\| \sum_{k=0}^{N} (T(k \cdot T_c) - (T_i - T_f) \cdot e^{-\frac{kT_c}{\tau}} - T_f) \right\|_2$$





Non-Linear models calibration



Many engineering design problems involve finding the best solution within certain constraints. Here's how it works:

1. **Defining the Objective**:

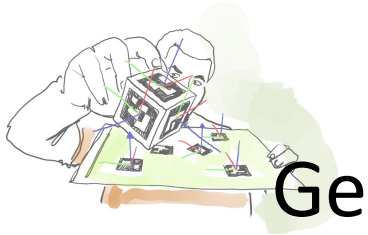
Every engineering task has a goal, such as minimizing cost, maximizing efficiency, or optimizing performance. This goal becomes the objective function in the optimization problem.

2. Identifying **Design Variables**:

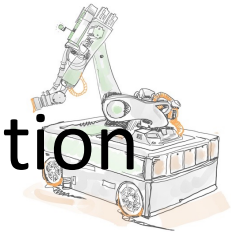
The design choices or parameters that can be adjusted (e.g., model parameters, dimensions of a structure, material properties, control inputs, controller parameters, relative motion, etc) are the decision variables. Let’s define them as “x”

3. **Formulating Constraints**:

Real-world problems are subject to limitations, such as physical laws, safety standards, resource availability, or budgetary constraints. These form the constraints in the optimization model.



Generalised problem solving with optimisation



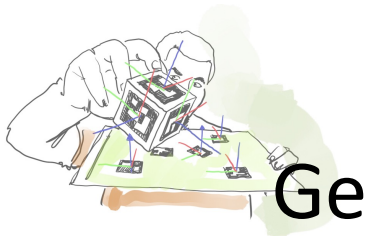
4. **Optimization Framework**:

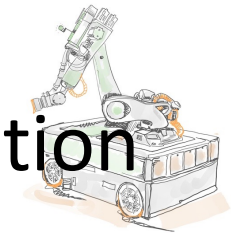
The problem is then mathematically expressed as:

Find x that Minimize f(x) subject to $g(x) \leq b$, $h(x)=c$

Where f(x) is the objective function, g(x) represents inequality constraints, and h(x) represents equality constraints.

"x" are the design variables.

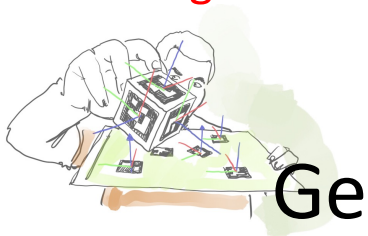




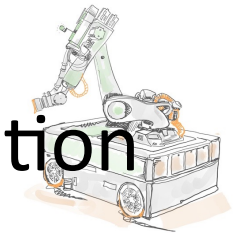
Examples:

- **Curve fitting**: fit experimental data with a curve, either in explicit or implicit form.
- **Motion estimation**: find the roto-translation between two lidar scans for example.
- **System identification**: Find the parameters of a dynamical system.
- **Structural Design**: Minimize material cost while ensuring strength and stability.
- **Control Design**: Find the best PID parameters to control a Plant. Or optimize the trajectory of a robot arm to minimize energy consumption while avoiding obstacles.
- **Circuit Design**: Maximize signal output while minimizing noise and power usage.

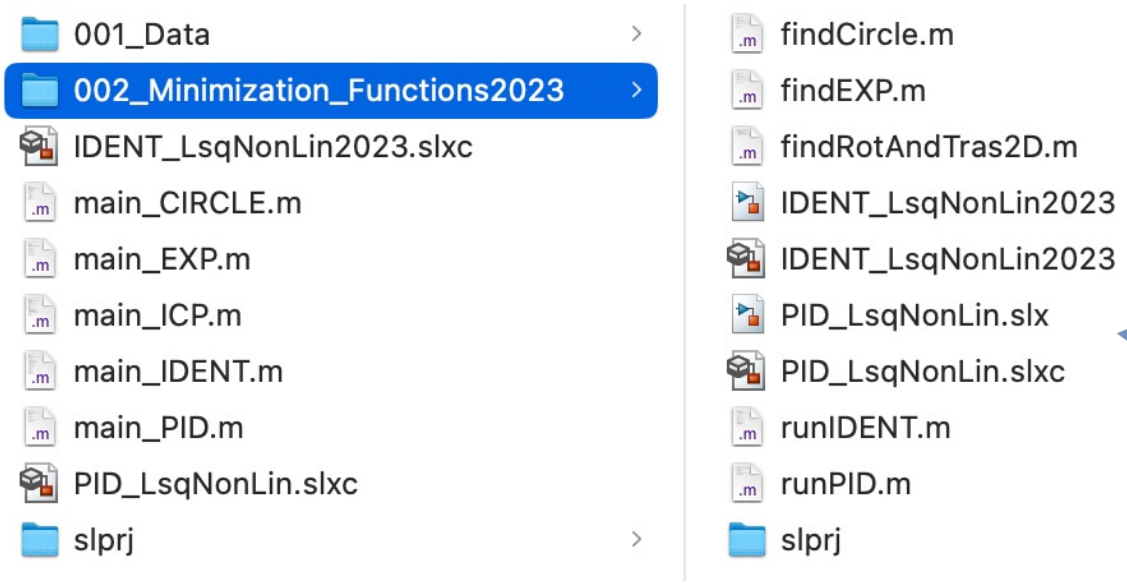
In conclusion, optimization frameworks unify engineering problems by allowing them to be solved systematically using similar mathematical tools and algorithms

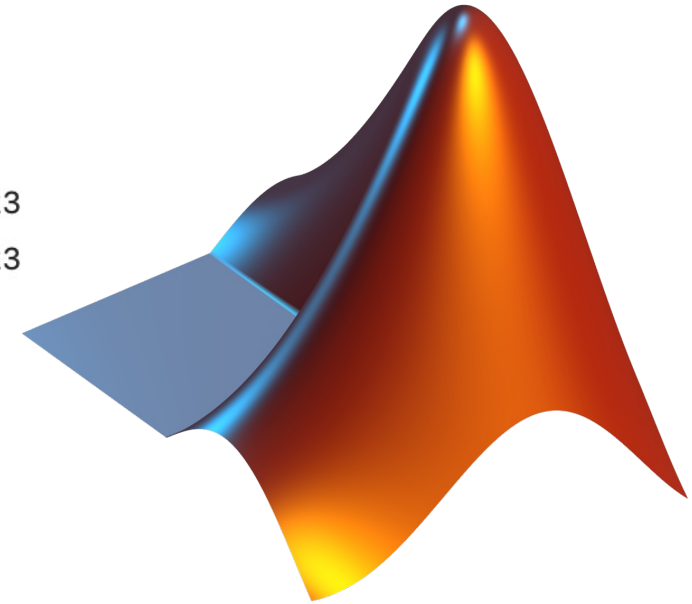


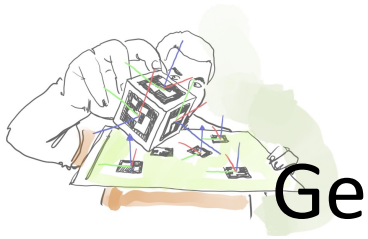
Generalised problem solving with optimisation



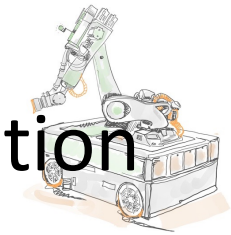
Let’s practice with matlab to understand how many problems can be solved with optimisation frameworks



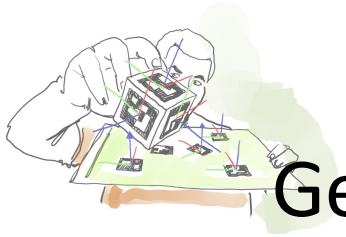




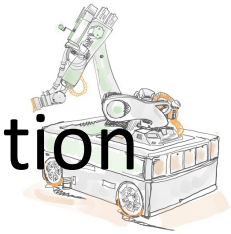
Generalised problem solving with optimisation



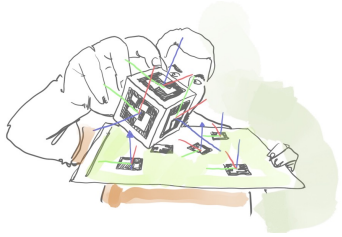
Here start a couple of lessons of practice on optimisation

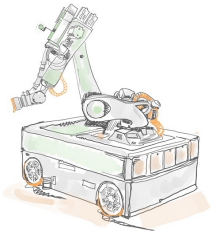


Generalised problem solving with optimisation



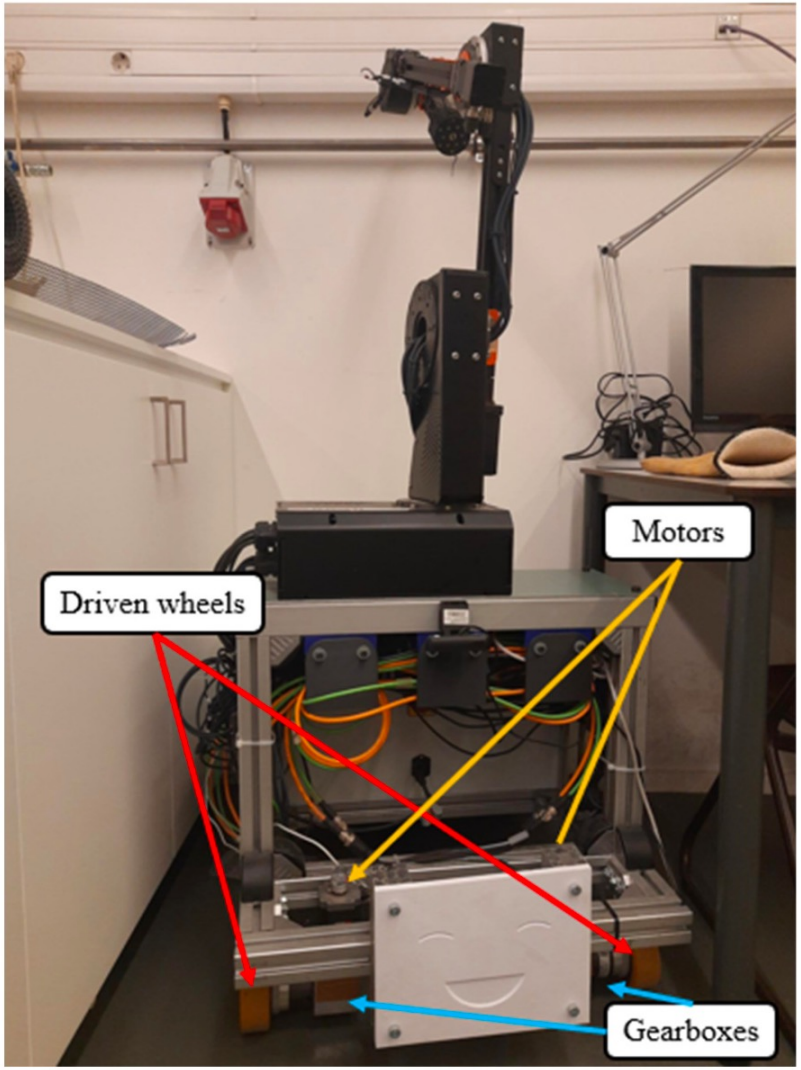
SOME CALIBRATION EXAMPLES IN LITERATURE

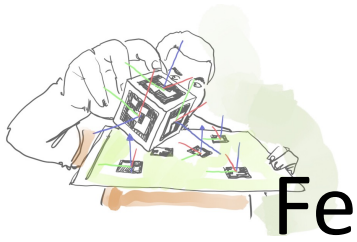




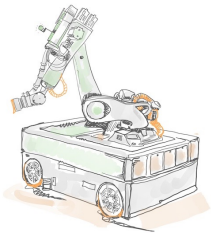
Differential drive kinematics and odometry for a mobile robot using TwinCAT

Miguel Ferreira, Luís Moreira and António Lopes

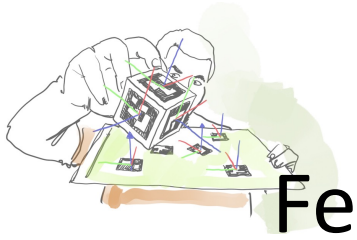




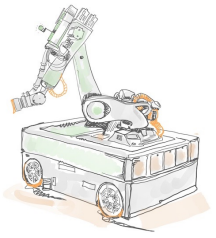
Ferreira, Moreira and Lopes

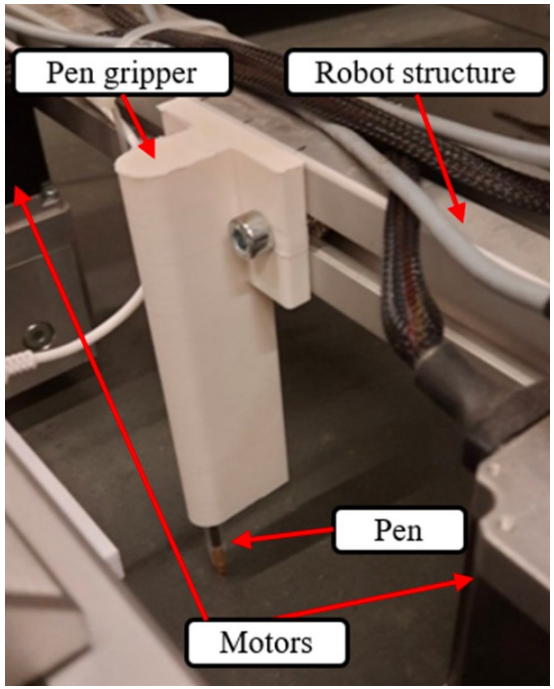


1. Before they correct for the linear motion (i.e. a Radius like index)



Ferreira, Moreira and Lopes





They inserted a pen on the robot to draw and measure the path:

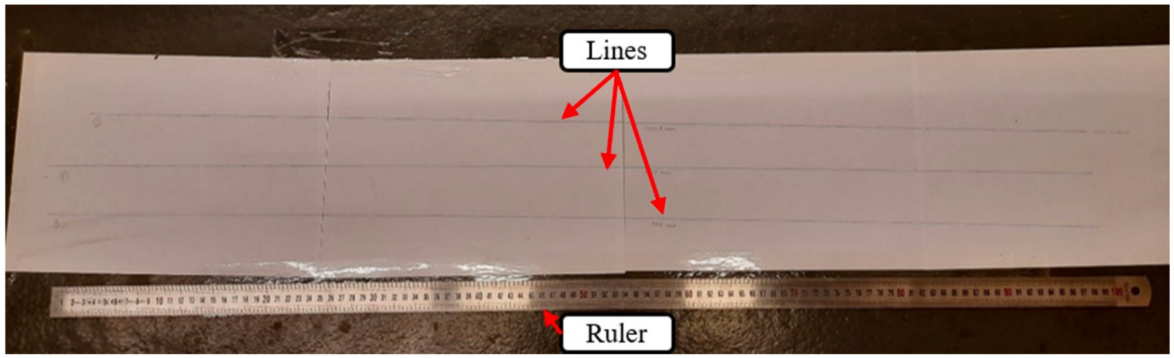
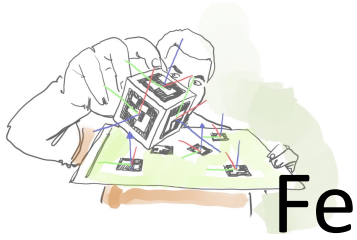


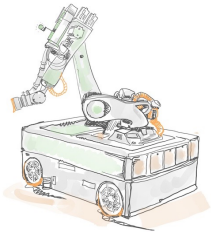
Figure 9. Picture of the lines produced by the robot and ruler used to measure them.

Table 1. Results of the initial linear odometry tests.

	x given by the odometry	Length of line	Error	%Error
Test 1	1000.0 mm	1009.5 mm	-9.5 mm	-0.94%
Test 2	1000.0 mm	1011.0 mm	-11.0 mm	-1.09%
Test 3	1000.0 mm	1010.5 mm	-10.5 mm	-1.04%
Average	1000.0 mm	1010.3 mm	-10.3 mm	-1.02%



Ferreira, Moreira and Lopes



$$SF = \frac{\pi \cdot D}{4 \cdot e} \cdot i$$

Eq 14

Why divided by 4?

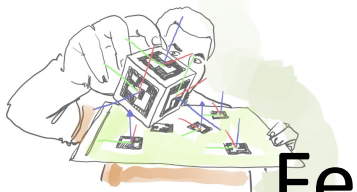
SF is the scaling factor, **D** is the diameter of the wheel, **i** is the gear ratio of the transmission system, and ***e*** is the number of encoder increments per rotation

$$SF_{corrected} = \frac{SF}{1+\%Error}$$

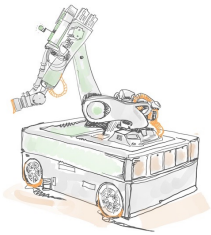
Eq 15

Why divided by (1+%Error)?

Since ***e*** and ***i*** are constant, **any changes to the SF can also be interpreted as changes to the wheel diameter** considered. The original SF used in this robot was 0.015340 mm/Inc (mm per increment of the encoder). This was obtained using a **nominal wheel diameter of 100 mm**. In this case, a corrected SF of 0.015498 mm/Inc is obtained. This equates to considering a wheel diameter of around **101.0 mm**



Ferreira, Moreira and Lopes



$$SF = \frac{\pi \cdot D}{\cancel{4} \cdot e} \cdot i = \frac{\text{Circumference}}{\text{Total encoder tics/revolution}}$$

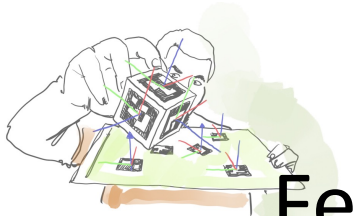
Why divided by 4? (an error?)

SF is the scaling factor
D is the diameter of the wheel
i is the gear ratio of the transmission system suppose = 1
e is the number of encoder increments per rotation
$\mathbf{n_{Tic}}$ is the number of encoder tics for a wheel along the manoeuvre

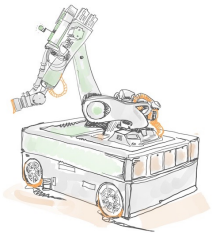
How we compute the scaling factor?

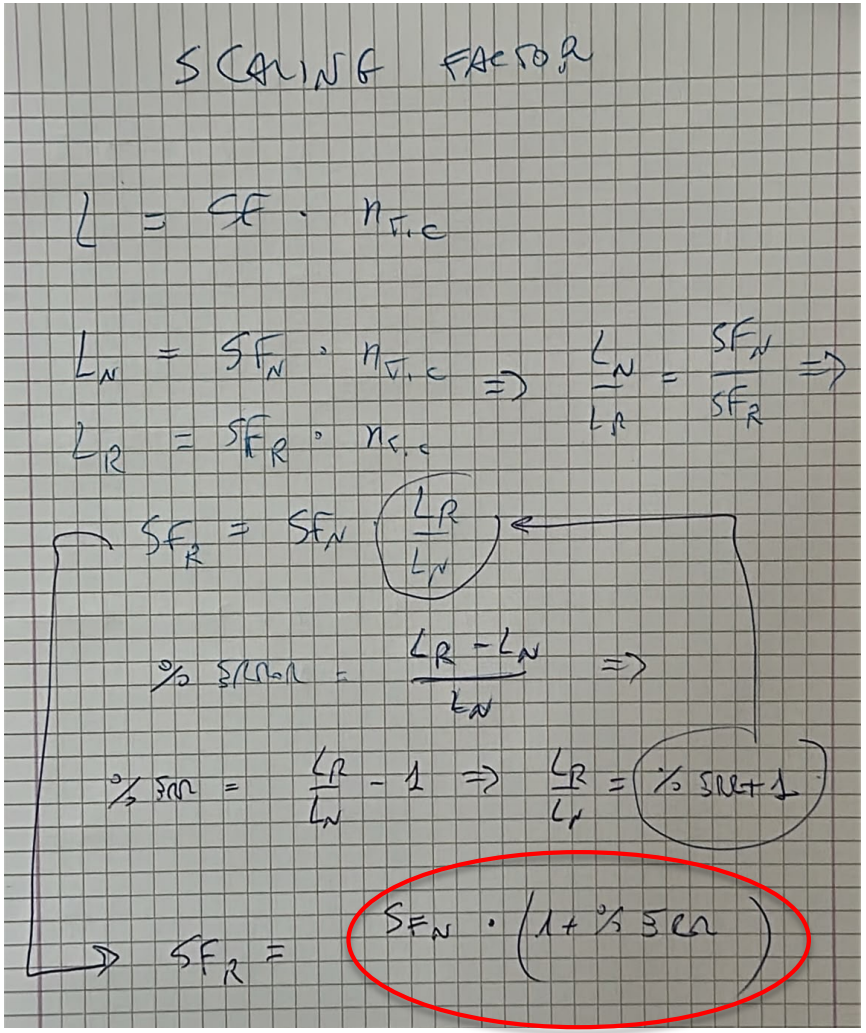
L = Length of a straight line path = $\frac{n_{Tic}}{e} \cdot 2\pi \cdot R$

That implies SF = $\frac{\pi D}{e} = \frac{L}{n_{Tic}}$



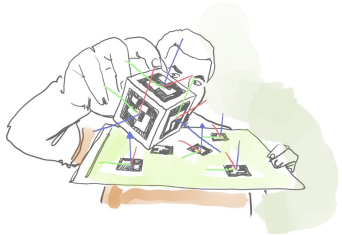
Ferreira, Moreira and Lopes

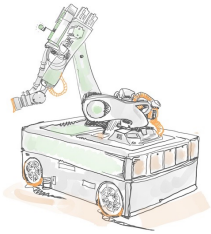




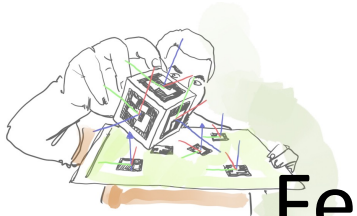
Why divided by (1+%Error)?

another error?

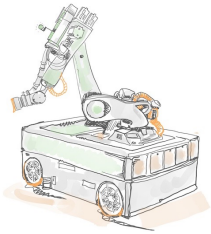




2. Then they correct for the interaxis B



Ferreira, Moreira and Lopes

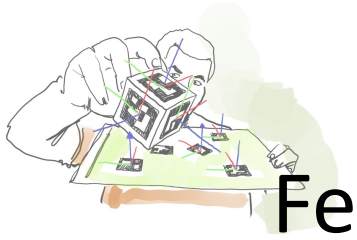


Angular odometry tests were performed after the correction of the linear odometry with the new scaling factor.

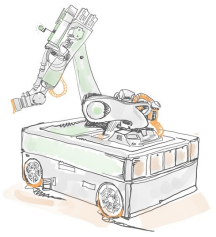
The robot was programmed to perform an in-place rotation of 360°. **A line parallel to the wheel's initial and final positions was drawn on a piece of paper** on the floor. The angle between these two lines was measured and compared to the angle given by the robot's odometry.
This test was done at an angular velocity of 0.182 rad/s and **repeated three times**

Table 3. Results of the first series of angular odometry tests.

	θ given by the odometry	Angle measured	Error	%Error
Test 1	360.0°	363.1°	-3.1°	-0.85%
Test 2	360.0°	362.9°	-2.9°	-0.80%
Test 3	360.0°	361.8°	-1.8°	-0.50%
Average	360.0°	362.6°	-2.6°	-0.72%



Ferreira, Moreira and Lopes

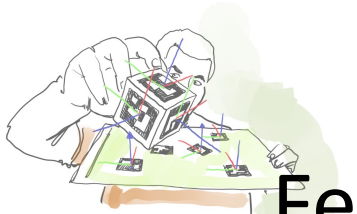


Considering that the linear odometry is already corrected, angular odometry can only be corrected by changing the value of the distance between wheels **B**, which is inversely proportional to the angular displacement.

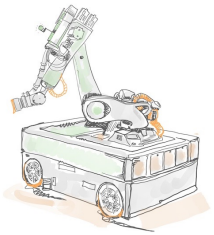
It can be corrected by:

$$B_{corrected} = B \cdot (1 + \%Error)$$ Eq 16

Why multiplied by (1+%Error)?



Ferreira, Moreira and Lopes



Considering that the linear odometry is already corrected, angular odometry can only be corrected by changing the value of the distance between wheels **B**, which is inversely proportional to the angular displacement

How can we compute B using the paper notations?

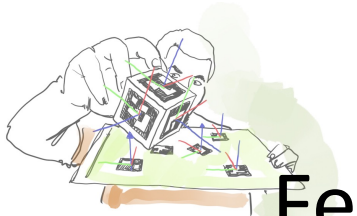
$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\delta} \end{bmatrix} = \begin{bmatrix} \frac{v_R + v_L}{2}\cos\delta \\ \frac{v_R + v_L}{2}\sin\delta \\ \frac{v_R - v_L}{b} \end{bmatrix} = \begin{bmatrix} v\cos\delta \\ v\sin\delta \\ \frac{v_R - v_L}{b} \end{bmatrix} \Rightarrow \delta_\theta = \frac{\delta_R - \delta_L}{B}$$

Where:

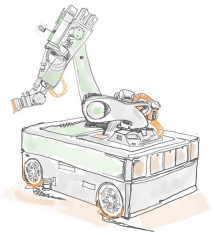
- δ_θ is the angular variation
- δ is the position variation $\delta = SF \bullet n_{Tic}$

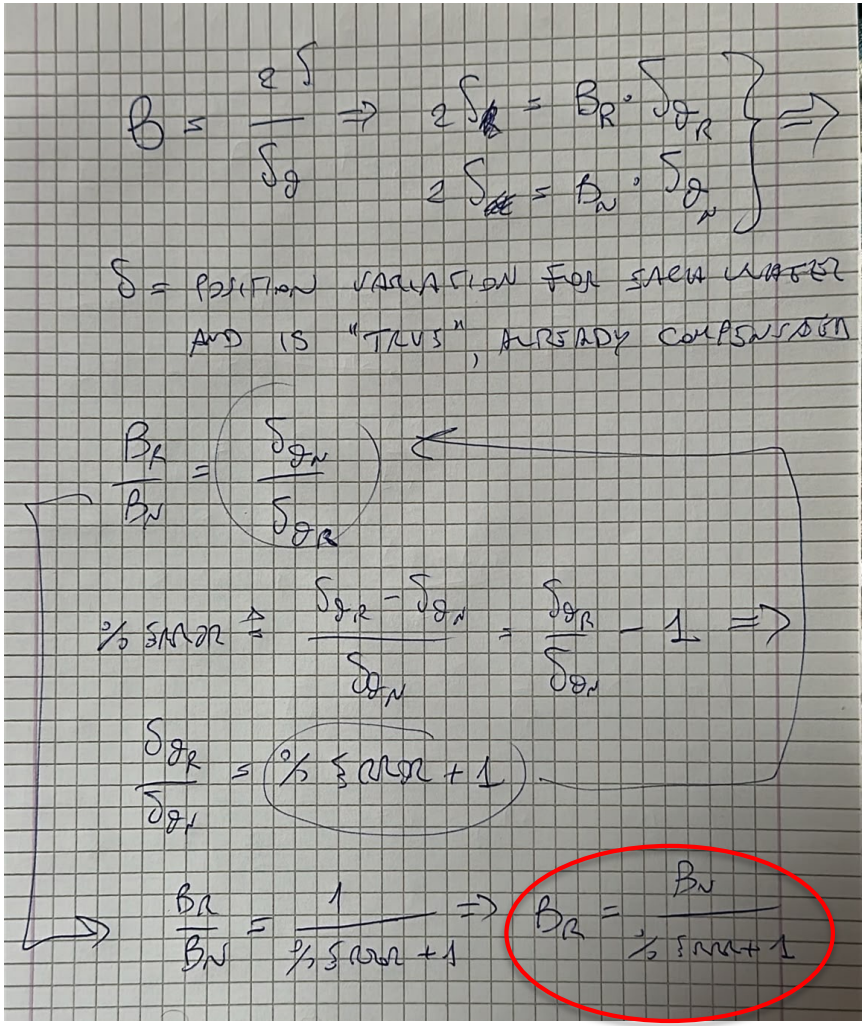
The robot was programmed to perform an in-place rotation of 360° means that the two wheels were programmed to perform an increment forward (right wheel) and backward (left) of the same amount δ:

That implies B = $\frac{2\delta}{\delta_\theta}$



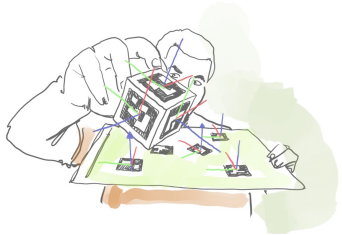
Ferreira, Moreira and Lopes

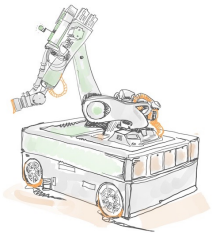




Why multiplied by (1+%Error)?

another error?



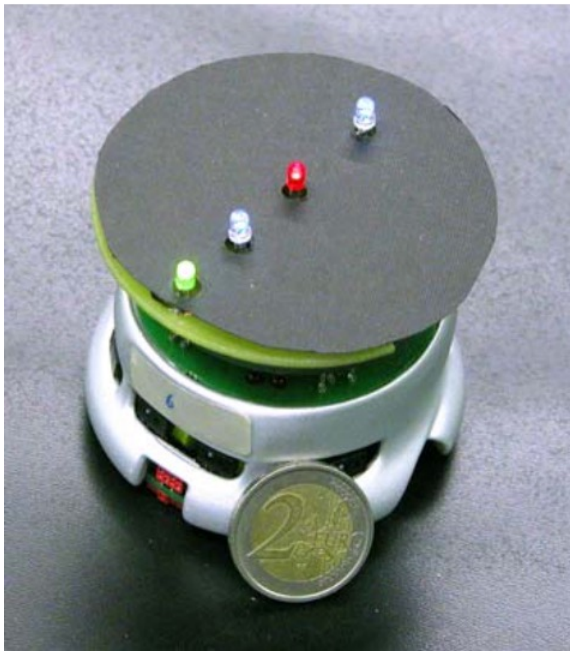


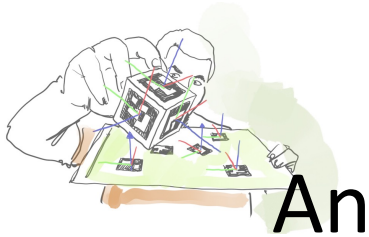
Linear estimation of the physical odometric parameters for differential-drive mobile robots

Gianluca Antonelli, Stefano Chiaverini

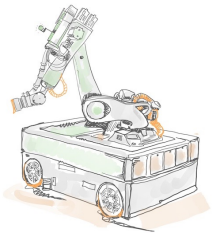
In this paper a calibration technique aimed at identifying the odometric parameters of differential-drive mobile robots is proposed. The algorithm is based on **two successive least-squares estimations** based on the continuous-time kinematic equations of motion; the **time-discretization error**, thus, is avoided …

No specific path is required. The basic technique makes use of video-camera measurements and angular readings of the wheels' encoders





Antonelli and Chiaverini



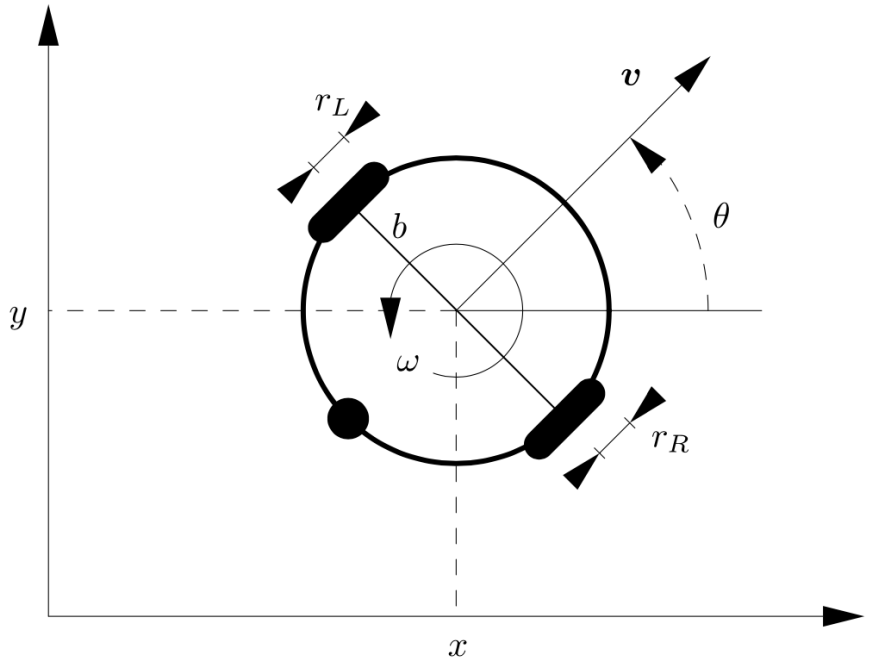
Definitions and algebraic facts:

$$\begin{cases} \dot{x} = v\cos(\theta), \\ \dot{y} = v\sin(\theta), \\ \dot{\theta} = \omega. \end{cases}$$

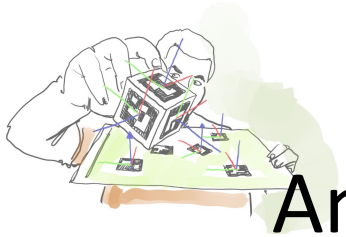
$$\begin{cases} v = \dfrac{r_R}{2}\omega_R + \dfrac{r_L}{2}\omega_L, \\ \omega = \dfrac{r_R}{b}\omega_R - \dfrac{r_L}{b}\omega_L, \end{cases}$$

$$\dot{s} = v.$$

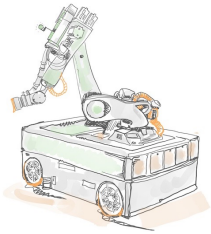
$$\dot{s} = \frac{r_R}{2}\omega_R + \frac{r_L}{2}\omega_L$$



The curvilinear velocity, and thus curvilinear increments, become a linear combination of the right and left wheel angular velocities as a function of two, unknown, parameters



Antonelli and Chiaverini



Definitions and algebraic facts:

Moreover, let us define

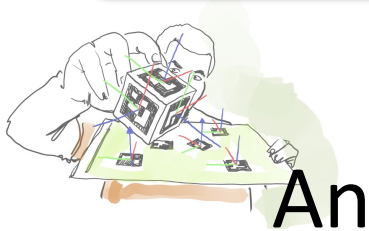
$$\alpha_R = \frac{r_R}{b},$$

$$\alpha_L = -\frac{r_L}{b},$$

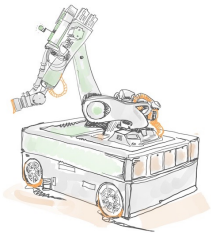
$$\omega = \frac{r_R}{b}\omega_R - \frac{r_L}{b}\omega_L,$$

$$\omega = \alpha_R\omega_R + \alpha_L\omega_L$$

That is, the angular velocity, and thus increments, become a linear combination of the right and left wheel angular velocities as a function of two, unknown, parameters



Antonelli and Chiaverini



The calibration procedure

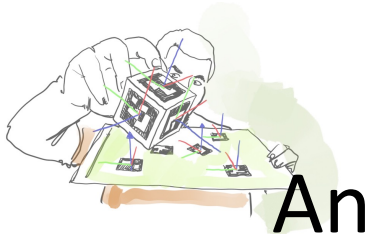
The proposed technique is based on the **continuous-time equations** by assuming that **a video-camera sensor is available** and the user can access the **encoder absolute positions**

Two successive least-squares operations are performed:

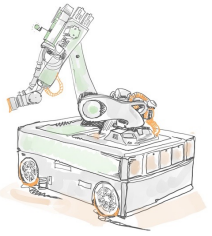
1. on the orientation equation and
2. on the curvilinear abscissa equation

1 $$\omega = \alpha_R \omega_R + \alpha_L \omega_L$$

2 $$\dot{s} = \frac{r_R}{2}\omega_R + \frac{r_L}{2}\omega_L$$



Antonelli and Chiaverini



By analytically integrating the orientation equation, one obtains

$$\omega = \alpha_R \omega_R + \alpha_L \omega_L$$

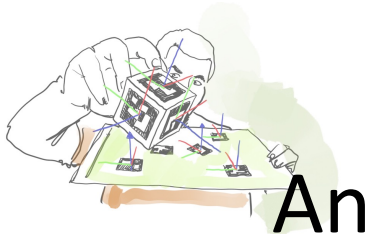
Integrating

$$\theta(t_2) - \theta(t_1) = \int_{t_1}^{t_2} \omega(\tau)\,\mathrm{d}\tau = \alpha_R \int_{t_1}^{t_2} \omega_R(\tau)\,\mathrm{d}\tau + \alpha_L \int_{t_1}^{t_2} \omega_L(\tau)\,\mathrm{d}\tau$$

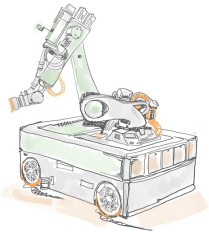
Considering a relative motion, i.e.:

- $\theta 1 = 0$
- $t1 = 0$
- $t2 = t$

$$\theta(t) = \alpha_R \theta_R(t) + \alpha_L \theta_L(t)$$



Antonelli and Chiaverini - orientation



$$\theta(t) = \alpha_R \theta_R(t) + \alpha_L \theta_L(t)$$

$$\theta_i = \theta(t_i)$$

$$\boldsymbol{\Phi}_{\theta_i} = [\theta_R(t_i) \quad \theta_L(t_i)]$$

$$\theta_i = \boldsymbol{\Phi}_{\theta_i} \begin{bmatrix} \alpha_R \\ \alpha_L \end{bmatrix}$$

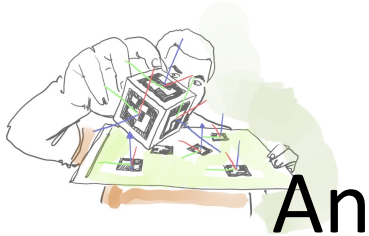
Measuring the robot's orientation with the camera it is possible to estimate the coefficients $\boldsymbol{\alpha_R}$ and $\boldsymbol{\alpha_L}$ by collecting a redundant set of data to work with an high-rectangular matrix Φ_θ: Px2

Measuring P samples, the equation becomes:

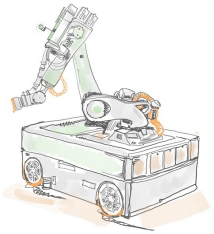
$$\begin{bmatrix} \theta_1 \\ \vdots \\ \theta_P \end{bmatrix} = \begin{bmatrix} \boldsymbol{\Phi}_{\theta_1} \\ \vdots \\ \boldsymbol{\Phi}_{\theta_P} \end{bmatrix} \begin{bmatrix} \alpha_R \\ \alpha_L \end{bmatrix} = \overline{\boldsymbol{\Phi}}_\theta \begin{bmatrix} \alpha_R \\ \alpha_L \end{bmatrix}$$

$$\begin{bmatrix} \hat{\alpha}_R \\ \hat{\alpha}_L \end{bmatrix} = (\overline{\boldsymbol{\Phi}}_\theta^{\mathrm{T}} \overline{\boldsymbol{\Phi}}_\theta)^{-1} \overline{\boldsymbol{\Phi}}_\theta^{\mathrm{T}} \begin{bmatrix} \theta_1 \\ \vdots \\ \theta_P \end{bmatrix}$$

The redundant linear equations optimisation leads to the pseudoinverse solution

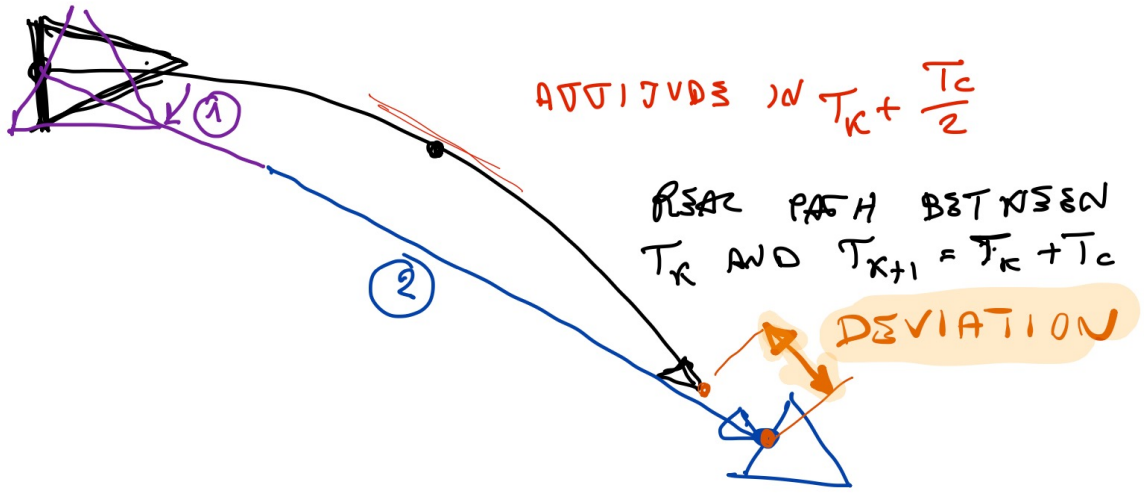


Antonelli and Chiaverini - orientation



To reduce the discretization error it is considered the half time attitude increment

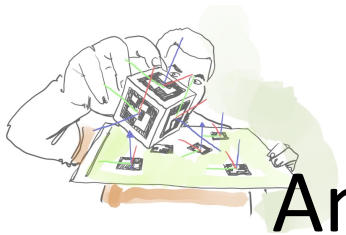
$$\begin{cases} x_{k+1} = x_k + T v_k \cos(\theta_k + T\omega_k/2) \\ y_{k+1} = y_k + T v_k \sin(\theta_k + T\omega_k/2), \\ \theta_{k+1} = \theta_k + T\omega_k, \end{cases}$$



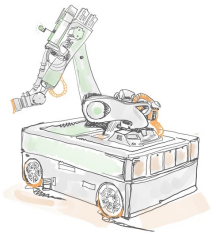
So, the strapdown procedure becomes:

1. Translate along a straight line aligned with the half-way attitude (tangent to the midpoint of the arc of circumference) → change **[x, y]**
2. Rotate the full amount of rotation at the end reaching the tangent attitude → change δ

In this way you will follow the chord!



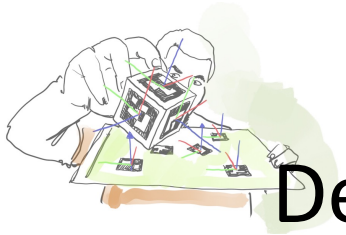
Antonelli and Chiaverini - motion



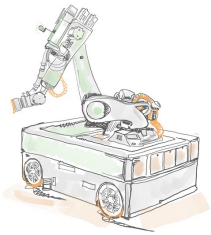
```
% kinematic simulation with l/r encoders Tic for each cycle
function SimulateEnc(obj, dt, NTic_r, NTic_l)
b = obj.K_param(1);
current_pose = obj.robotCurrentPose;

Vr = (obj.K_param(2) + obj.DK_param(2)) * 2*pi * NTic_r /
(obj.Enc_res * dt);
Vl = (obj.K_param(3) + obj.DK_param(3)) * 2*pi * NTic_l /
(obj.Enc_res * dt);

% Diff Drive Model:
% [   xdot    ]   [ cos(delta)/2      cos(delta)/2 ]  [  Vr  ]
% [   ydot    ] = [ sin(delta)/2      sin(delta)/2 ]  [  Vl  ]
% [ deltadot  ]   [     1/b              -1/b ]
J = [cos(current_pose(3))/2 cos(current_pose(3))/2;...
     sin(current_pose(3))/2 sin(current_pose(3))/2;...
     1/b                    -1/b];
derivative_pose = J * [Vr; Vl];
xdot = derivative_pose(1);
ydot = derivative_pose(2);
deltadot = derivative_pose(3);
```



Deviation in simulation - strapdown



1

```
x_new = current_pose(1) + xdot * dt;
y_new = current_pose(2) + ydot * dt;
```

2

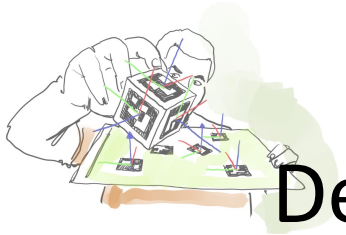
```
delta_new = current_pose(3) + deltadot * dt;

obj.robotCurrentPose = [x_new, y_new, delta_new];
obj.actualPath = [obj.actualPath; x_new, y_new];

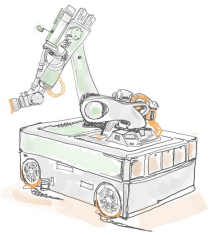
end
```

1: In computing the new position [x y] uses the actual (old) pose (Delta)

2: the new pose (Delta) is computed in this step but used to estimate position in the following cycle

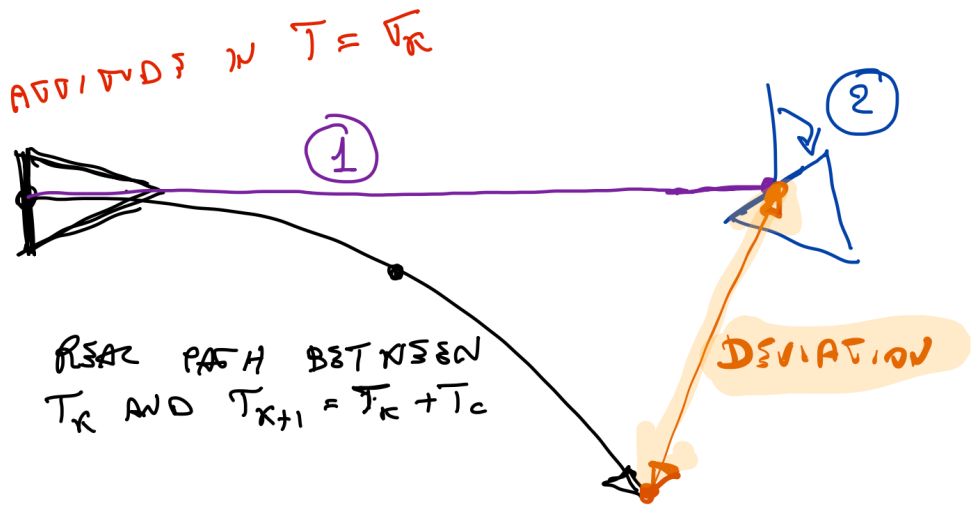


Deviation in simulation - strapdown



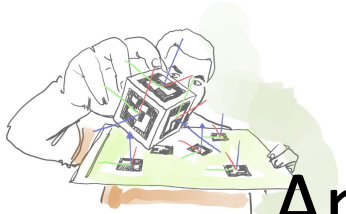
In our simplified discrete time simulator we, instead, considered the attitude at the instant ***K*** to compute the increment in position

$$\begin{cases} x_{k+1} = x_k + T v_k \cos(\theta_k + T\omega_k/2) \\ y_{k+1} = y_k + T v_k \sin(\theta_k + T\omega_k/2) \\ \theta_{k+1} = \theta_k + T\omega_k, \end{cases}$$

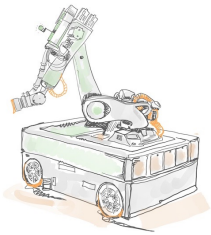


So, the strapdown procedure was:

1. Translate along a straight line as long as the curvilinear coordinate
2. Rotate the robot along the full attitude variation in the time step



Antonelli and Chiaverini



Recalling:

$$\alpha_R = \frac{r_R}{b}, \qquad \alpha_L = -\frac{r_L}{b},$$

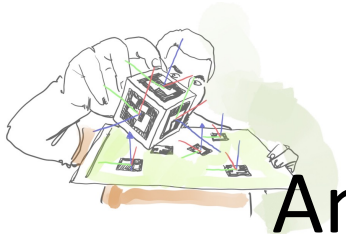
$$r_R = \frac{\alpha_R}{\alpha_L} \cdot r_L$$

$$v = \frac{r_R}{2}\omega_R + \frac{r_L}{2}\omega_L$$

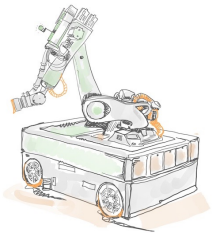
$$v = \left(-\frac{\alpha_R}{2\alpha_L} r_L \omega_{R,k} + \frac{r_L}{2}\omega_{L,k}\right)$$

Same kinematic equations in discrete form:

$$\begin{cases} x_{k+1} = x_k + T v_k \cos(\theta_k + T\omega_k/2) \\ y_{k+1} = y_k + T v_k \sin(\theta_k + T\omega_k/2) \\ \theta_{k+1} = \theta_k + T\omega_k, \end{cases}$$

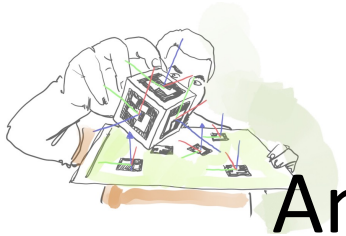


Antonelli and Chiaverini

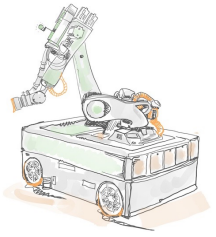


$$x_{k+1} = x_k + T\left(-\frac{\alpha_R}{2\alpha_L} r_L \omega_{R,k} + \frac{r_L}{2}\omega_{L,k}\right) \cos(\theta_k + T\omega_k/2)$$

$$y_{k+1} = y_k + T\left(-\frac{\alpha_R}{2\alpha_L} r_L \omega_{R,k} + \frac{r_L}{2}\omega_{L,k}\right) \sin(\theta_k + T\omega_k/2)$$



Antonelli and Chiaverini



Measured by camera

Already estimated

Measured by encoder

The rotational dynamics was already calibrated

$$\begin{bmatrix} x_{k+1} - x_k \\ y_{k+1} - y_k \end{bmatrix} = \frac{T}{2} \begin{bmatrix} (-\frac{\alpha_R}{\alpha_L}\omega_{R,k} + \omega_{L,k}) \cos(\theta_k + T\omega_k/2) \\ (-\frac{\alpha_R}{\alpha_L}\omega_{R,k} + \omega_{L,k}) \sin(\theta_k + T\omega_k/2) \end{bmatrix} r_L$$

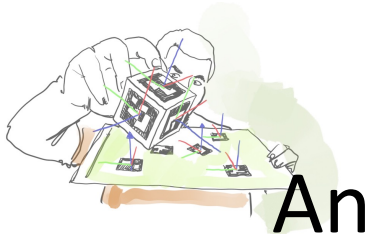
c_i stands for $c_i = \cos(\theta_i + T\omega_i/2)$

$$s_i = \sin(\theta_i + T\omega_i/2)$$

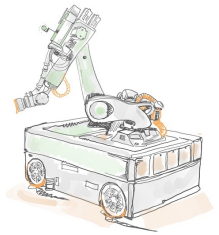
$$\begin{bmatrix} x_K - x_0 \\ y_K - y_0 \end{bmatrix} = \frac{T}{2} \begin{bmatrix} -\frac{\alpha_R}{\alpha_L} \sum_{i=0}^{K-1} \omega_{R,i} c_i + \sum_{i=0}^{K-1} \omega_{L,i} c_i \\ -\frac{\alpha_R}{\alpha_L} \sum_{i=0}^{K-1} \omega_{R,i} s_i + \sum_{i=0}^{K-1} \omega_{L,i} s_i \end{bmatrix} r_L$$

It is then possible to **execute P different trajectories**, collect the required data and **by resorting again to the least squares algorithm,** compute r_L

$$\Delta xy = \Phi \cdot r_L \quad \Rightarrow \quad r_L = (\Phi^T \Phi)^{-1} \Phi^T \cdot \Delta xy$$

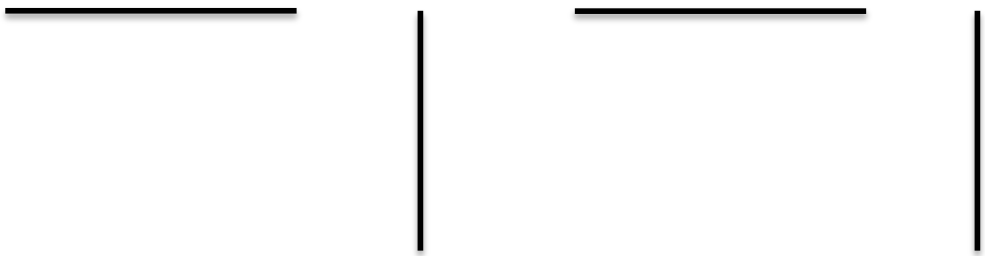


Antonelli and Chiaverini

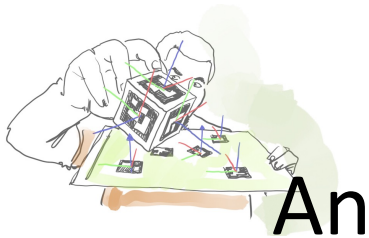


Note:

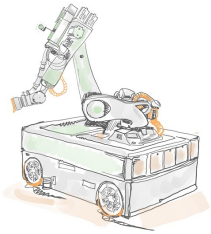
$$r_L = (\Phi^T \Phi)^{-1} \Phi^T \cdot \Delta xy$$



=

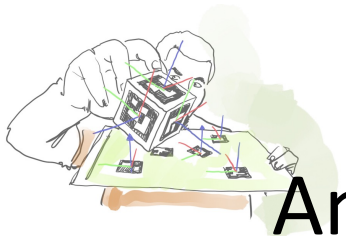


Antonelli and Chiaverini

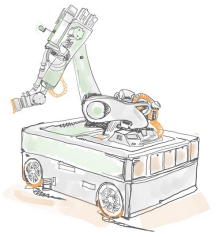


We have estimated $\begin{bmatrix} \hat{\alpha}_R \\ \hat{\alpha}_L \end{bmatrix}$ and $\hat{r}_L$

$$\hat{r}_R = -\frac{\hat{\alpha}_R}{\hat{\alpha}_L}\hat{r}_L \quad \Rightarrow \quad \hat{b} = \frac{\hat{r}_R}{\hat{\alpha}_R}$$



Antonelli and Chiaverini



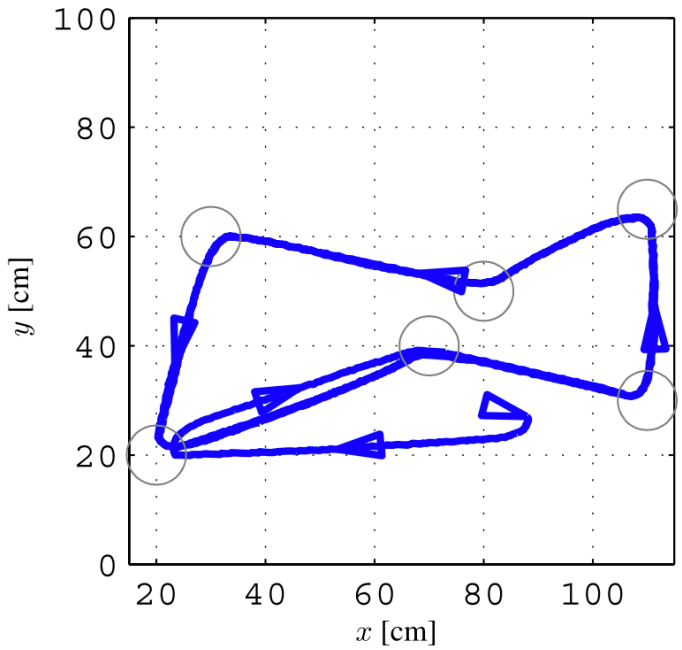


Fig. 3 Path of the identification trajectory with the via-points evidenced. The starting point and the via-points are randomly selected.

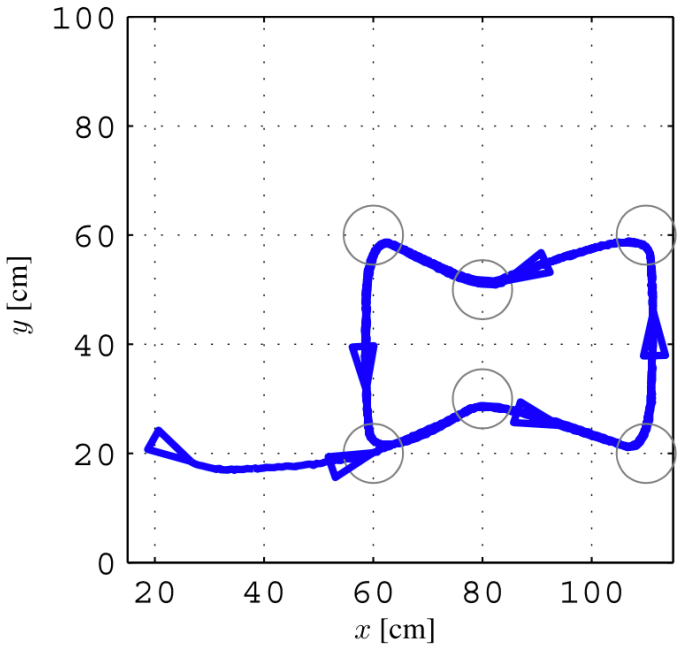
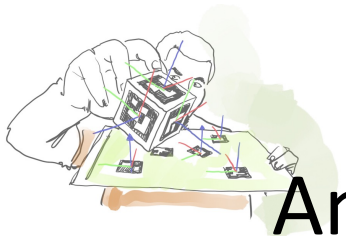


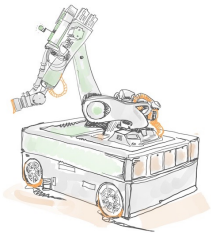
Fig. 8 Path of the validation trajectory with the via-points evidenced. As for the identification trajectory, the via-points and the starting position are randomly selected

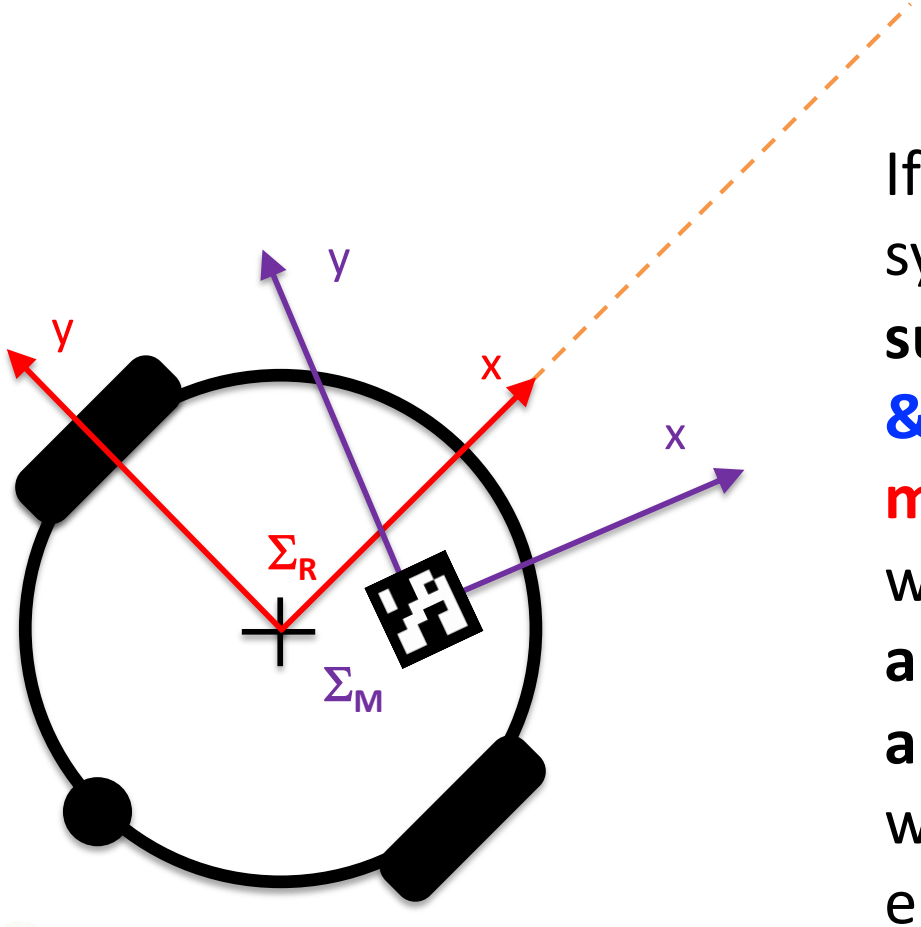
The trajectory to calibrate (identify) parameters

The trajectory to validate the results --> **must be different!**

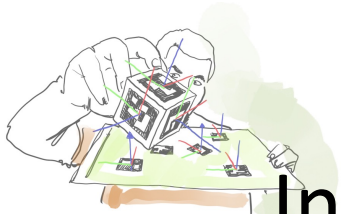


Antonelli and Chiaverini

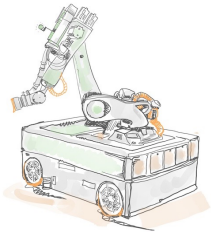




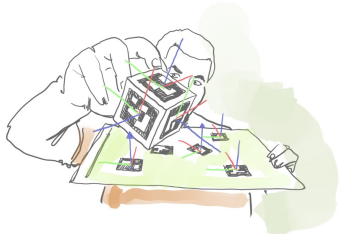
If we install a measurement system onboard the robot **such as the LED of Antonelli & Chiaverini or an ARUCO marker** that we can track with a video camera **we are able to follow its position and attitude** in real time with respect to the environment.

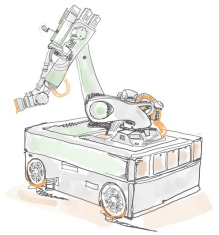


Installation of a measurement system



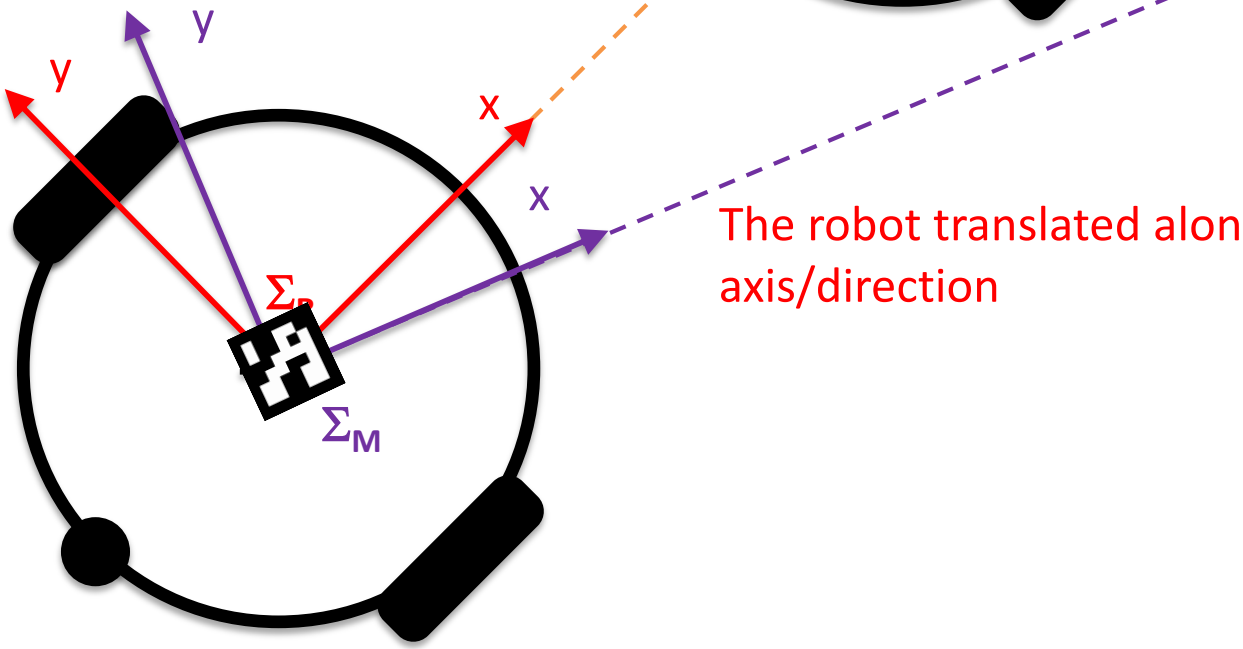
But…. To add a measurement system is ‘gratis’????



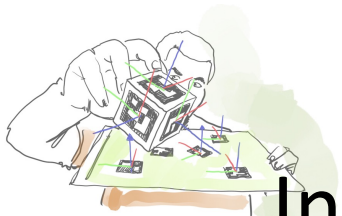


Effect of misalignment -
The target (or LED) rotation is unknown and believed to be zero

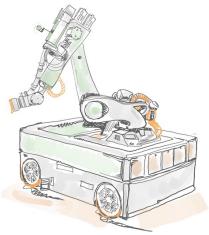
Ideal case: equal wheel radii + same encoder Tics



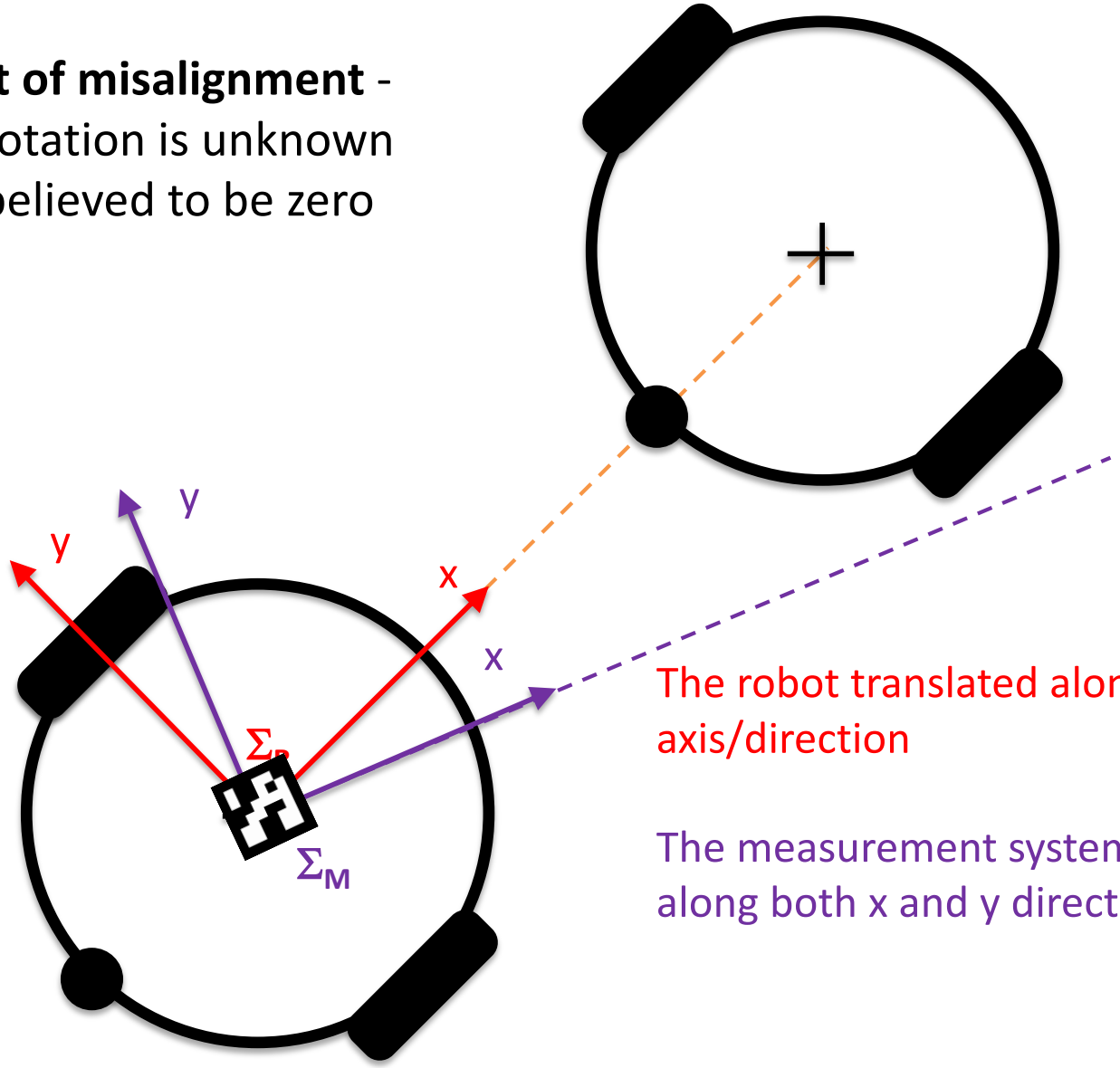
The robot translated along its forward axis/direction



Installation of a measurement system

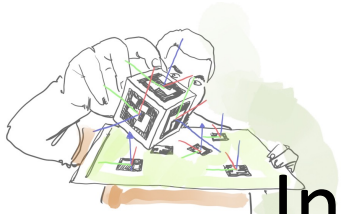


Effect of misalignment -
The rotation is unknown and believed to be zero

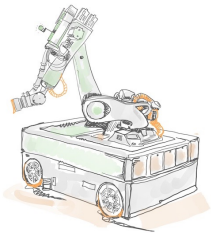


The robot translated along its forward axis/direction

The measurement system perceived a motion along both x and y directions!!!

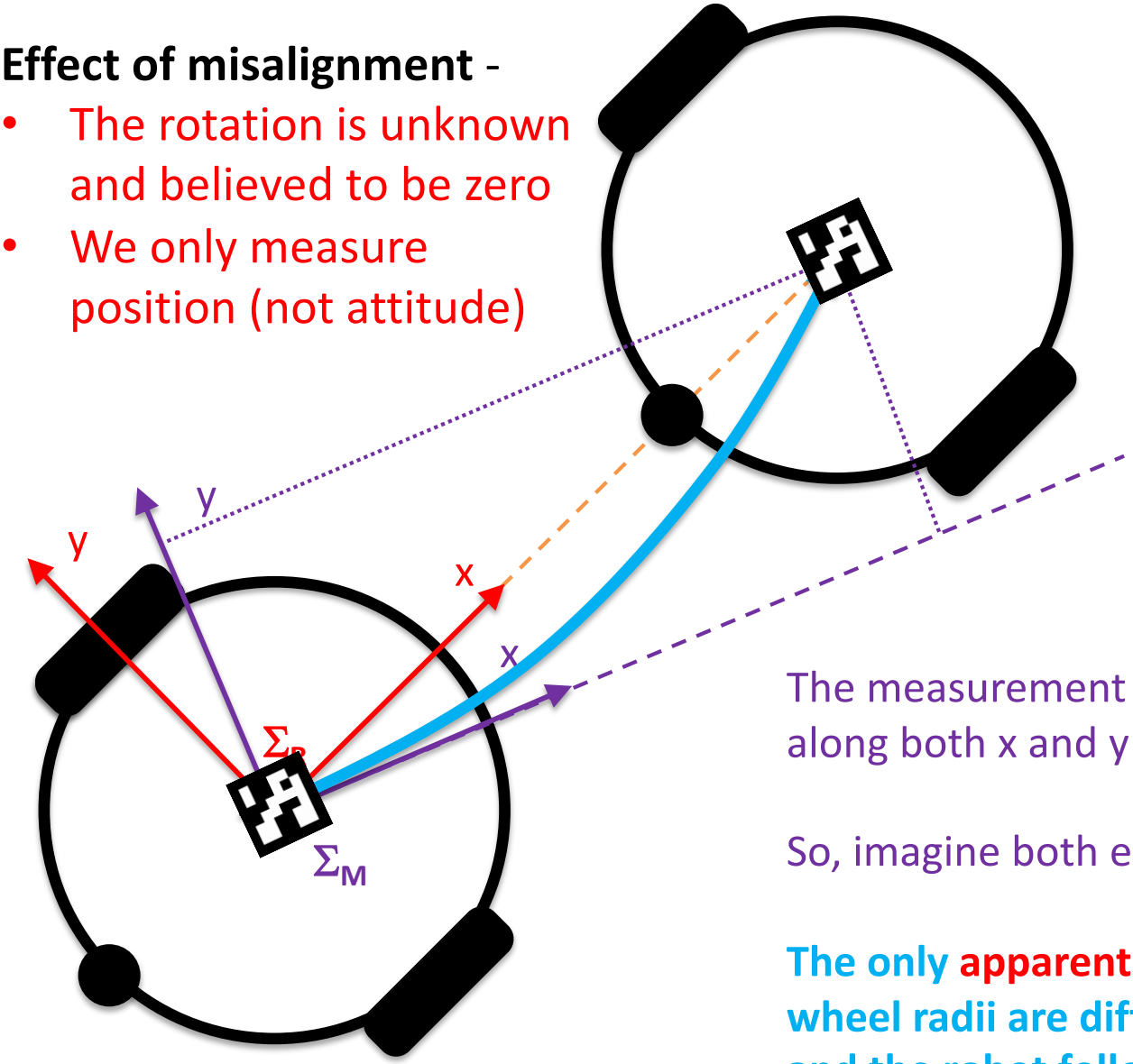


Installation of a measurement system



Effect of misalignment -

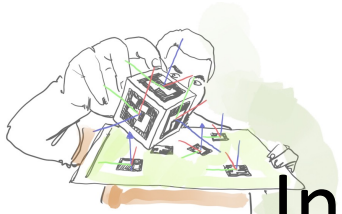
- The rotation is unknown and believed to be zero
- We only measure position (not attitude)



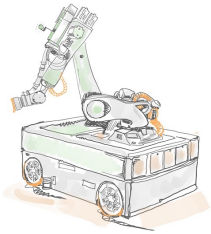
The measurement system perceived a motion along both x and y directions

So, imagine both encoder velocities are equal

The only apparent solution is that the two wheel radii are different (the left one smaller) and the robot followed an arc of circle

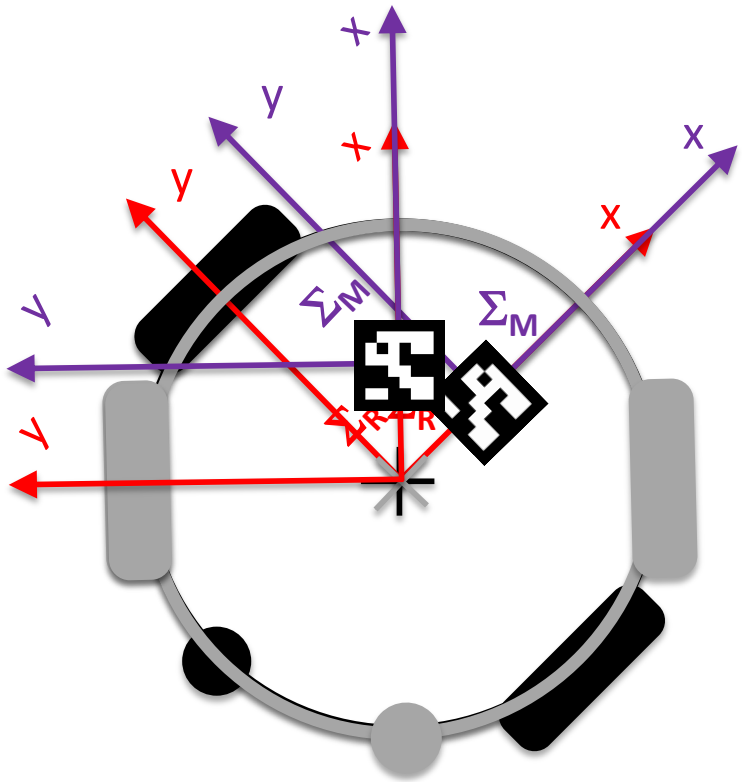


Installation of a measurement system



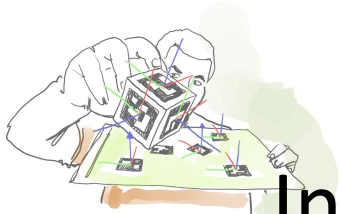
Effect of a bias wrt the robot control point - Rotation in place

Ideal case: equal wheel radii + opposite encoder Tics (opposite velocities)

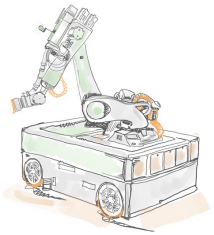


The robot only rotated in place

The measurement system perceived a roto-translation!!!

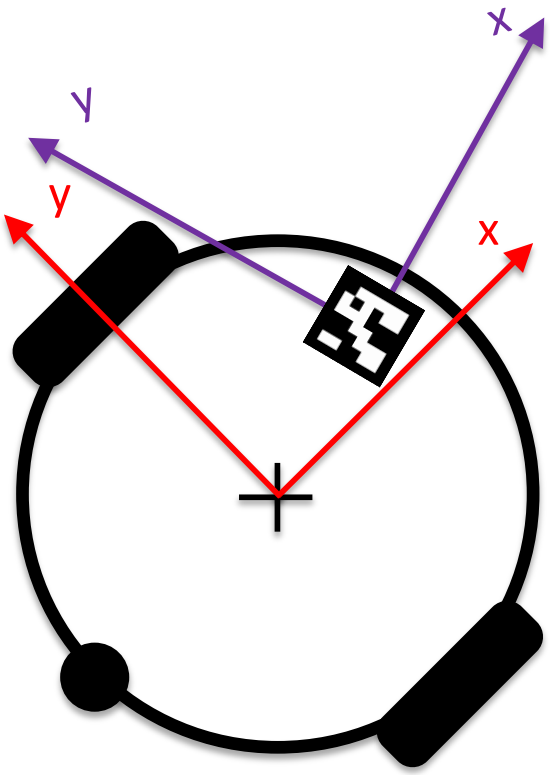


Installation of a measurement system

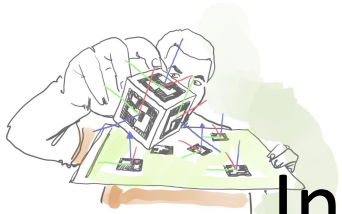


So, we believed to solve a problem adding a measurement system BUT we also added another problem:

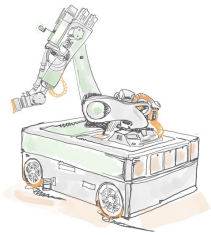
- The unknown **position** and **orientation** of the measurement system wrt the robot reference system: 3 more unknown parameters



Extrinsic Robot-Sensor Calibration



Installation of a measurement system



Simultaneous Calibration of Odometry and Sensor Parameters for Mobile Robots

Andrea Censi, Antonio Franchi, Luca Marchionni and Giuseppe Oriolo

Let $q = (q_x, q_y, q_\theta)$ be the robot pose with respect to a fixed world frame

For a differential-drive robot, the pose evolves according to the differential equation:

$$\dot{\boldsymbol{q}} = \begin{pmatrix} \cos q_\theta & 0 \\ \sin q_\theta & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} v \\ \omega \end{pmatrix}.$$

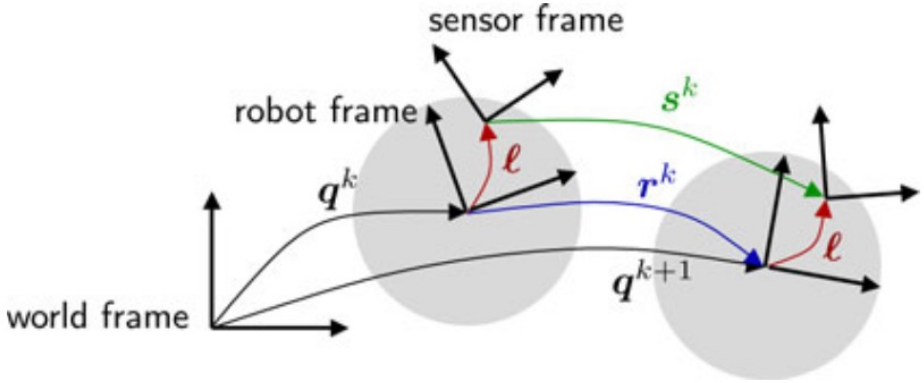
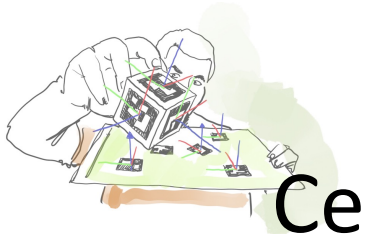
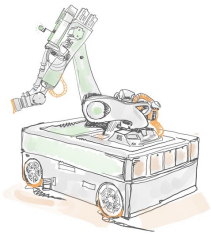


Fig. 1. Robot pose is $\boldsymbol{q}^k \in \mathrm{SE}(2)$ with respect to the world frame; the sensor pose is $\boldsymbol{\ell} \in \mathrm{SE}(2)$ with respect to the robot frame; $\boldsymbol{r}^k \in \mathrm{SE}(2)$ is the robot displacement between poses; and $\boldsymbol{s}^k \in \mathrm{SE}(2)$ is the displacement seen by the sensor in its own reference frame.



Censi, Franchi, Marchionni and Oriolo

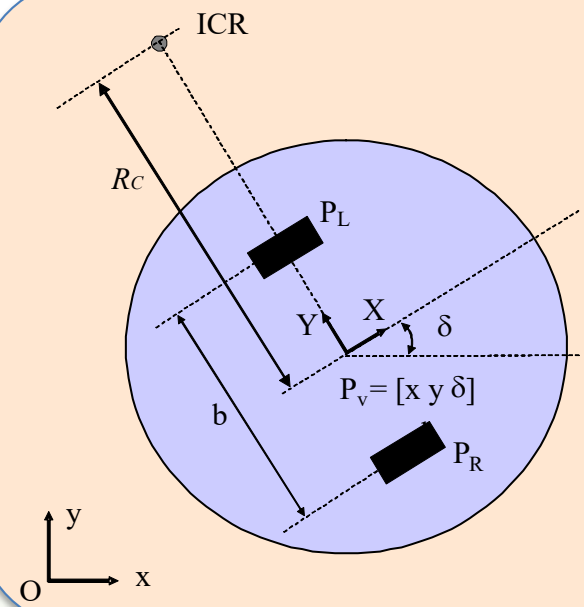


The driving velocity **v** and the steering velocity **ω** depend on the left and right wheel velocities ω_L , ω_R by a linear transformation:

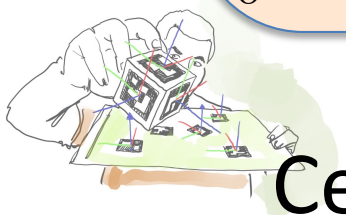
$$\begin{pmatrix} v \\ \omega \end{pmatrix} = \boldsymbol{J} \begin{pmatrix} \omega_{\mathrm{L}} \\ \omega_{\mathrm{R}} \end{pmatrix}$$

$$\boldsymbol{J} = \begin{pmatrix} J_{11} & J_{12} \\ J_{21} & J_{22} \end{pmatrix} = \begin{pmatrix} +r_{\mathrm{L}}/2 & +r_{\mathrm{R}}/2 \\ -r_{\mathrm{L}}/b & +r_{\mathrm{R}}/b \end{pmatrix}$$

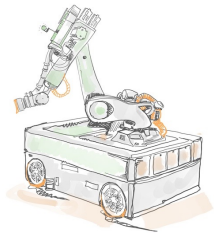
Are the same relations already introduced ($\omega r = v$)



$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\delta} \end{bmatrix} = \begin{bmatrix} \frac{v_R + v_L}{2}\cos\delta \\ \frac{v_R + v_L}{2}\sin\delta \\ \frac{v_R - v_L}{b} \end{bmatrix} = \begin{bmatrix} v\cos\delta \\ v\sin\delta \\ \frac{v_R - v_L}{b} \end{bmatrix}$$

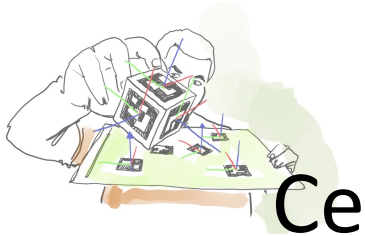


Censi, Franchi, Marchionni and Oriolo

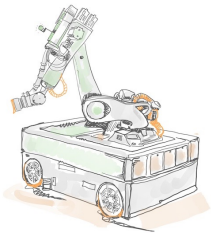


Calibration parameters to be estimated	
$r_{\mathrm{R}}, r_{\mathrm{L}}$	wheel radii
b	distance between wheels
$\boldsymbol{\ell}$	sensor pose relative to robot frame
Robot kinematics	
$\boldsymbol{q}$	robot pose relative to world frame
$\omega_{\mathrm{L}}, \omega_{\mathrm{R}}$	left/right wheel velocity
v, ω	driving/steering robot velocities
$\boldsymbol{J}$	linear map between wheel and robot velocities
Sensing process	
$\boldsymbol{m}^k$	exteroceptive measurements, available at time t_k
$\boldsymbol{r}^k$	robot displacement in the k-th interval $[t_k, t_{k+1}]$
$\boldsymbol{s}^k$	sensor displacement in the k-th interval
$\hat{\boldsymbol{s}}^k$	sensor displacement estimated from $\boldsymbol{m}^k$ and $\boldsymbol{m}^{k+1}$
$\boldsymbol{\nu}$	sensor velocity in the sensor frame

No, this is the case of a Car-Like model



Censi, Franchi, Marchionni and Oriolo



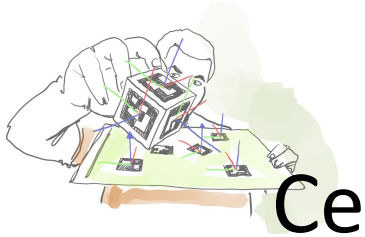
We assume that it is possible to estimate the sensor's egomotion $\boldsymbol{s_k}$ given the exteroceptive measurements $\boldsymbol{m_k}$ and $\boldsymbol{m_{k+1}}$, and we call $\boldsymbol{s}\hat{}$ such estimate

For example, if the sensor is a range- finder, the egomotion can be estimated via scan matching, if it is a 3D camera that acquires a marker on the robot the relative pose is directly estimated

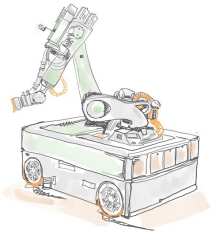
Simultaneous calibration:
Given:

- the wheel velocities $\omega_L(t)$ and $\omega_R(t)$ for $t \in [t1, tn]$,
- the estimated sensor egomotion $\hat{s}_k$ $(k = 1, \ldots, n-1)$ corresponding to the exteroceptive observations at times $t_1 < \cdots < t_k < \cdots < t_n$

Find: the maximum-likelihood estimate for the parameters r_L, r_R, b, l_x, l_y, and l_θ



Censi, Franchi, Marchionni and Oriolo



The same as Antonelli & Chiaverini!!!

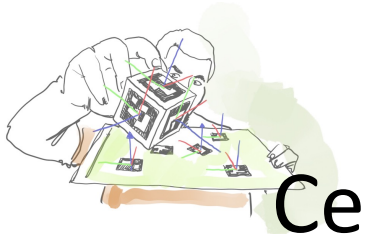
1 - Data collection

2- Linear estimation of J_{21} and J_{22} measuring N attitude variations

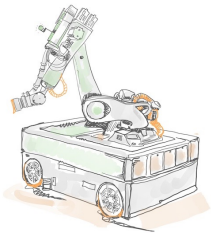
3 – Non-Linear estimation of the other 4 parameters measuring N pose variations

Algorithm 1 Simultaneous calibration of odometry and sensor parameters

1) Passively collect measurements over any sufficiently exciting trajectory.
2) For each interval, run the sensor displacement algorithm to obtain the estimates $\hat{s}^k$.
   Each interval thus contributes the data sample
   $$\langle \hat{s}^k, \omega_L(t), \omega_R(t) \rangle, \qquad t \in [t_k, t_{k+1}].$$
3) Repeat N times (for outlier rejection):
   Linear estimation of J_{21}, J_{22}:
   a) For all samples, compute the matrix $\boldsymbol{L}_k$ using (10).
   b) Form the matrix $\sum_k \boldsymbol{L}_k^T \boldsymbol{L}_k$. If the condition number of this matrix is over a threshold, declare the problem underconstrained and stop.
   c) Compute J_{21}, J_{22} using (11).
   Nonlinear estimation of the calibration parameters:
   d) For all samples, compute c_x^k, c_y^k (19–20) and $\boldsymbol{Q}_k$ using (22).
   e) Let $\boldsymbol{M} = \sum_k \boldsymbol{Q}_k^T \boldsymbol{Q}_k$.
   f) Compute the coefficients a, b, c using to (28–30) and find the two candidates $\lambda^{(1)}, \lambda^{(2)}$.
   g) For each $\lambda^{(i)}$:
      i) Compute the 5×5 matrix $\boldsymbol{N}^{(i)} = \boldsymbol{M} - \lambda^{(i)} \boldsymbol{W}$.
      ii) If the rank of $\boldsymbol{N}^{(i)}$ is less than 4, declare the problem underconstrained and stop.
      iii) Find a vector $\boldsymbol{\gamma}^{(i)}$ in the kernel of $\boldsymbol{N}^i$.
      iv) Compute $\boldsymbol{\varphi}^{(i)}$ using (31).
   h) Choose the optimal $\boldsymbol{\varphi}$ between $\boldsymbol{\varphi}^{(1)}$ and $\boldsymbol{\varphi}^{(2)}$ by computing the objective function.
   i) Compute the other parameters using (32).
   Outlier rejection:
   j) Compute the χ-value of each sample using (33).
   k) Discard a fraction α of samples with the highest χ.



Censi, Franchi, Marchionni and Oriolo

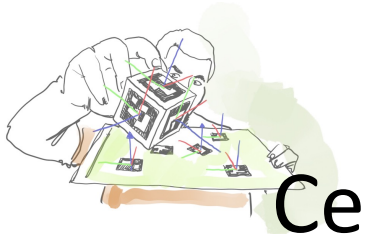


You can use:

- Antonelli & Cjiaverini for the 2 parameters
- a single optimization without all the steps of Censi & All to find the 4 remaining parameters

OR

use a single optimization without all the steps of Censi & All to find all the 6 parameters



Censi, Franchi, Marchionni and Oriolo

